

Microcontroladores: Laboratorio 2

1st Hector Pereira

Ingeniería en Mecatrónica

Universidad Tecnológica (UTEC)

Fray Bentos, Uruguay

hector.pereira@estudiantes.utec.edu.uy

2nd Isaac Martirena

Ingeniería en Mecatrónica

Universidad Tecnológica (UTEC)

Fray Bentos, Uruguay

isaac.martirena@estudiantes.utec.edu.uy

Resumen—El presente informe describe el desarrollo de un conjunto de sistemas embebidos implementados sobre el microcontrolador ATmega328P, en el marco del curso de Microcontroladores. Cada módulo diseñado aborda distintos aspectos de la arquitectura AVR y de la programación en lenguaje C, integrando tanto periféricos digitales como analógicos. Los proyectos incluyen un *plotter* cartesiano controlado por servomotores, un sistema de detección y visualización de colores mediante sensor LDR y tira LED WS2812, un piano electrónico con reproducción de melodías, y una cerradura digital con almacenamiento de contraseñas en EEPROM y teclado matricial. A través de estas implementaciones se consolidaron competencias en el manejo de interrupciones, temporizadores, comunicación serial, control por PWM y diseño de máquinas de estado, logrando una comprensión integral del funcionamiento coordinado entre hardware y software en sistemas embebidos.

Keywords: *microcontroladores, ATmega328P, plotter, sistemas embebidos, PWM, EEPROM, UART, sensores, servomotores, interfaz usuario-sistema, procesamiento de imágenes*

I. INTRODUCCIÓN

El presente laboratorio se desarrolla en el marco del curso de Microcontroladores, teniendo como objetivo principal la aplicación práctica de los conocimientos teóricos adquiridos sobre la arquitectura AVR y la programación en lenguaje C. A diferencia de los ejercicios iniciales, orientados al manejo individual de periféricos, esta instancia propone el diseño e integración de varios módulos funcionales que operan en un entorno de tiempo real, permitiendo comprender la interacción entre hardware y software dentro de un sistema embebido.

I-A. Propósito del laboratorio

El propósito central es diseñar, programar e integrar subsistemas independientes sobre el microcontrolador ATmega328P, abordando diversas interfaces de entrada y salida, comunicación serial, control de actuadores por *Pulse Width Modulation* (PWM), adquisición analógica de señales y almacenamiento no volátil mediante memoria EEPROM. A través de este trabajo se busca consolidar la capacidad de desarrollar proyectos embebidos que respondan de forma

determinista a eventos externos y gestionen múltiples tareas concurrentes con recursos limitados.

I-B. Objetivos

Objetivo general:

- Integrar y programar distintos módulos electrónicos basados en el microcontrolador ATmega328P, aplicando conceptos de arquitectura AVR, control de periféricos y desarrollo de sistemas embebidos en tiempo real.

Objetivos específicos:

- Implementar un sistema de control cartesiano (*plotter*) mediante servomotores y comunicación serial.
- Desarrollar un sistema de detección de colores que combine medición analógica, control PWM y visualización RGB.
- Diseñar un piano electrónico con reproducción de notas y melodías almacenadas utilizando temporizadores e interrupciones.
- Construir una cerradura digital que permita el ingreso y cambio de contraseñas almacenadas en la memoria EEPROM.
- Aplicar principios de modularidad y diseño estructurado en C para lograr una integración eficiente entre hardware y software.

I-C. Descripción general de los módulos

El laboratorio se divide en cuatro implementaciones principales:

- **Plotter:** sistema de control cartesiano para el movimiento coordinado de ejes y el trazado de figuras mediante servomotores.
- **Colores:** sensor de detección cromática basado en una fotocelda (LDR), control de servomotor para posicionamiento angular y visualización del color identificado en una tira LED WS2812.
- **Piano:** instrumento electrónico que reproduce notas musicales y melodías almacenadas en memoria utilizando temporizadores e interrupciones.
- **Cerradura electrónica:** sistema de acceso protegido mediante contraseña, con interfaz de teclado matricial, pantalla LCD y almacenamiento persistente en EEPROM.

I-D. Competencias desarrolladas

Durante el desarrollo de los módulos se aplicaron y fortalecieron competencias clave en el ámbito de la ingeniería mecatrónica, tales como:

- Programación estructurada en lenguaje C sobre microcontroladores de arquitectura AVR.
- Configuración y uso de temporizadores e interrupciones para la ejecución de tareas simultáneas.
- Aplicación de PWM en el control de servomotores y generación de señales de audio.
- Implementación de comunicación serial (USART) para intercambio de datos y monitoreo del sistema.
- Manejo de memoria EEPROM para almacenamiento no volátil de información.
- Diseño de máquinas de estados para la gestión de la interfaz usuario-sistema.

II. MARCO TEÓRICO

II-A. Procesamiento de señales y sensores de color

Un sensor de luz o *Light Dependent Resistor* (LDR) es un dispositivo fotoresistivo cuya resistencia eléctrica disminuye cuando aumenta la intensidad luminosa incidente. Este comportamiento se debe a que la radiación electromagnética libera portadores de carga en el material semiconductor —generalmente sulfuro de cadmio— reduciendo su resistividad [1]. De este modo, el LDR convierte la variación de luz en una señal eléctrica analógica que puede ser interpretada por el convertidor analógico-digital (ADC) del microcontrolador.

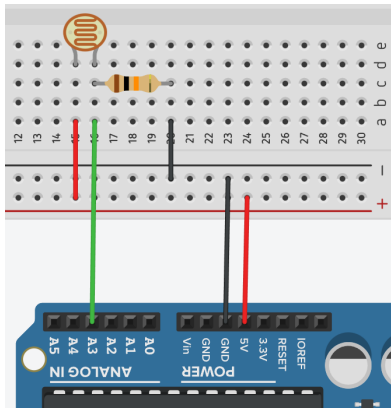


Figura 1. Esquema de conexión del sensor LDR en un divisor resistivo cuya salida se dirige al canal ADC del microcontrolador ATmega328P. Fuente: <https://paraarduino.com/sensores/fotorresistencias/valor-maximo-de-una-fotorresistencia/>.

En el caso del ATmega328P, el ADC posee una resolución de 10 bits, lo que permite representar las señales analógicas dentro de un rango discreto de 0 a 1023, donde los valores extremos corresponden a la tensión mínima y máxima de referencia [2]. Estos niveles cuantificados pueden relacionarse con la intensidad relativa de los componentes del modelo aditivo de color **RGB** (*Red*, *Green*, *Blue*), el cual describe los colores como combinaciones de tres luces primarias. En dicho modelo, cada componente adopta un

valor dentro de una escala de 0 a 255, rango que se utiliza en la representación digital del color en dispositivos emisores de luz, como pantallas o tiras de LEDs [3].

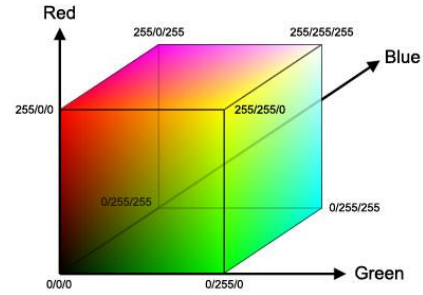


Figura 2. Representación tridimensional del modelo de color RGB, donde cada eje corresponde a una de las componentes primarias (*Red*, *Green*, *Blue*) y su combinación genera el espacio de colores aditivos. Fuente: https://www.researchgate.net/figure/a-RGB-Color-Space-7-b-YCbCr-Color-Space-8_fig1_298734907.

Dado que la respuesta espectral del LDR no es lineal ni uniforme frente a distintas longitudes de onda, la interpretación cromática requiere un proceso de calibración que establezca correspondencias entre los valores eléctricos medidos y los tonos del espacio RGB. Este principio permite compensar las variaciones inherentes del sensor y asegurar una medición coherente en condiciones de iluminación variables.

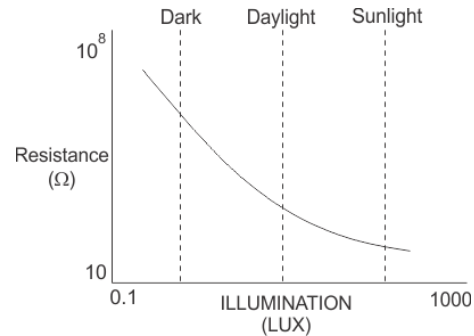


Figura 3. Curva típica de resistencia frente a la iluminación en un sensor LDR. Se observa el comportamiento no lineal del dispositivo, cuya resistencia disminuye de forma exponencial al aumentar la intensidad luminosa. Fuente: https://www.researchgate.net/figure/Resistance-vs-illumination-curve-LDRs-are-light-dependent-devices-whose-resistance_fig3_318029856.

II-B. Señales PWM y control de servomotores

El *Pulse Width Modulation* (PWM) o modulación por ancho de pulso es una técnica utilizada en sistemas embebidos para generar señales de control analógicas a partir de salidas digitales [3]. Consiste en emitir una señal periódica de frecuencia constante cuyo ciclo de trabajo (*duty cycle*) varía en función de la magnitud deseada. Este ciclo se define como la relación entre el tiempo en nivel alto (t_{on}) y el periodo total (T):

$$D = \frac{t_{on}}{T} \times 100 (\%)$$

En los servomotores de tipo *hobby*, el control de posición se basa en la variación del ancho del pulso dentro de un periodo fijo de aproximadamente 20 ms, correspondiente a una frecuencia de 50 Hz [4]. Pulsos de alrededor de 1 ms y 2 ms suelen representar los límites de desplazamiento angular del eje (por ejemplo, 0° y 180°). Así, la posición del servo se determina proporcionalmente al tiempo que la señal permanece en nivel alto.

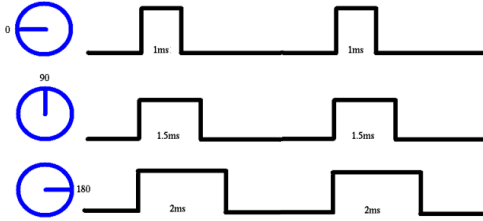


Figura 4. Señal PWM utilizada para el control de un servomotor de tipo *hobby*. La variación del ancho de pulso dentro de un periodo fijo de 20 ms determina la posición angular del eje, donde pulsos de aproximadamente 1 ms, 1.5 ms y 2 ms corresponden a 0°, 90° y 180°, respectivamente. Fuente: <https://electronics.stackexchange.com/questions/346603/driving-servo-motor-with-pwm-signal>.

En el microcontrolador ATmega328P, la generación de señales PWM se realiza mediante los temporizadores de hardware (*Timer/Counter*) [2]. Estos permiten obtener pulsos de duración precisa sin requerir intervención constante del procesador, lo que garantiza estabilidad temporal, repetitividad y eficiencia computacional. Al delegar la modulación al hardware, el microcontrolador puede atender otras tareas concurrentes sin afectar la calidad de la señal de control.

II-C. Reproducción de sonido digital

El sonido es una onda mecánica periódica cuya frecuencia determina la percepción del tono musical. En términos físicos, la frecuencia (f), medida en hertzios (Hz), representa el número de oscilaciones por segundo, mientras que su inverso define el periodo ($T = 1/f$) [1]. En los sistemas electrónicos, este principio se aplica mediante la generación de señales eléctricas alternantes que hacen vibrar un transductor, como un buzzer piezoeléctrico, produciendo así ondas sonoras audibles.

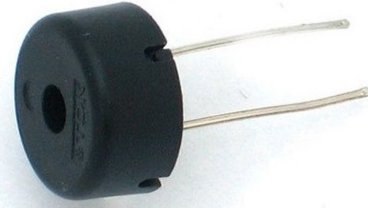


Figura 5. Buzzer piezoeléctrico modelo PS1240, utilizado como transductor acústico para la generación de sonido mediante señales cuadradas. Su funcionamiento se basa en la vibración mecánica del material piezoeléctrico al aplicarse una señal alternante. Fuente: https://www.pishop.us/product/pi-ezo-buzzer-ps1240/?srsltid=AfmBOooScrJtIiyvIt6zUSB8B502aj-LKG3PUMQgUtYcYVKXt_-yDs2.

En los microcontroladores, la síntesis de sonido se logra a través de la modulación temporal de señales digitales, comúnmente en forma de ondas cuadradas. La frecuencia de conmutación de estas ondas determina el tono percibido, mientras que su duración controla la extensión temporal de la nota [3]. De esta manera, es posible representar una escala musical a partir de un conjunto discreto de frecuencias asociadas a las notas estándar.

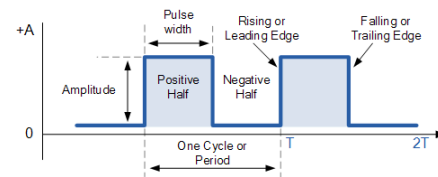


Figura 6. Onda cuadrada periódica empleada para la generación de tonos audibles en un buzzer piezoeléctrico. Se indican el periodo (T) y la frecuencia ($f = 1/T$), parámetros fundamentales que determinan el tono y la duración de la nota reproducida. Fuente: <https://www.allcoelec.com/blog/on-sinusoidal-waveforms-square,rectangular-and-pulsed-waveforms.html>.

En el ámbito de la música digital, el formato **MIDI** (*Musical Instrument Digital Interface*) constituye un estándar ampliamente utilizado para describir eventos musicales mediante códigos numéricos [5]. Cada evento MIDI contiene información como la nota a ejecutar, su duración, intensidad y canal asignado. Aunque no transmite sonido directamente, este formato permite definir secuencias musicales que pueden ser interpretadas por distintos dispositivos electrónicos, incluidos los microcontroladores, mediante la generación de las frecuencias correspondientes.

La reproducción de melodías digitales se basa, por tanto, en la secuenciación de notas definidas por su frecuencia y duración. Este enfoque, derivado de los principios del formato MIDI, permite sintetizar melodías simples mediante

la temporización de señales cuadradas, demostrando la capacidad de los sistemas embebidos para generar sonido a partir de parámetros discretos de control temporal y frecuencia.

II-D. Interfaz de usuario e interacción hombre-máquina

En los sistemas embebidos, la **interfaz de usuario** (*User Interface*, UI) se define como el conjunto de componentes físicos y lógicos que facilitan la comunicación entre el usuario y el dispositivo. Su función principal es transformar las acciones del operador en respuestas visuales o auditivas que reflejen el estado interno del sistema, permitiendo un control intuitivo y seguro del mismo [6].

Este intercambio bidireccional se logra a través de distintos medios de entrada y salida. Los periféricos de entrada —como teclados, pulsadores o sensores— registran las acciones del usuario, mientras que los de salida —como pantallas, indicadores luminosos o buzzers— proporcionan *feedback* sobre el resultado de dichas acciones. En conjunto, estos elementos conforman un canal de comunicación que mejora la interacción y la comprensión del funcionamiento del sistema [3].

Para organizar la lógica de funcionamiento, los sistemas embebidos suelen emplear una **máquina de estados finitos** (FSM, por sus siglas en inglés). Este modelo conceptual divide la operación del programa en un conjunto finito de estados definidos, entre los cuales se producen transiciones en respuesta a eventos determinados [7]. Dicho enfoque favorece la modularidad, la claridad del código y la predictibilidad del comportamiento, cualidades esenciales en el diseño de interfaces de usuario para entornos con recursos limitados.

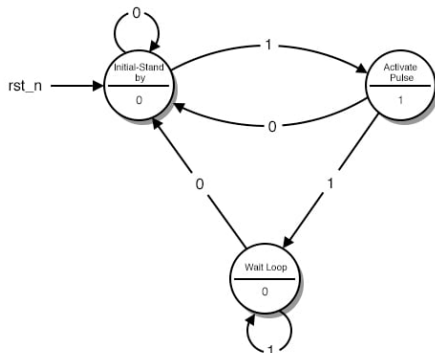


Figura 7. Ejemplo de diagrama de una máquina de estados finitos (FSM), donde cada nodo representa un estado lógico del sistema y las transiciones se activan por eventos o condiciones externas. Este enfoque permite modelar el comportamiento secuencial y predecible de una interfaz de usuario embebida. Fuente: <https://www.allaboutcircuits.com/textbook/digital/chpt-11/finite-state-machines/>.

II-E. Almacenamiento no volátil y EEPROM

La **EEPROM** (*Electrically Erasable Programmable Read-Only Memory*) es un tipo de memoria no volátil que permite conservar la información almacenada incluso después de interrumpir la alimentación del sistema [3]. A

diferencia de la memoria RAM, que se borra al apagar el dispositivo, la EEPROM mantiene los datos de forma permanente hasta que son modificados explícitamente por el programa, lo que la hace ideal para guardar configuraciones o valores críticos de usuario.

En el microcontrolador ATmega328P, este tipo de memoria tiene una capacidad de 1 kB organizada en celdas de 8 bits, cada una de las cuales puede ser escrita y borrada eléctricamente un número limitado de veces —del orden de 10^5 ciclos— según las especificaciones del fabricante [2]. Debido a esta limitación, es recomendable minimizar las operaciones de escritura para evitar el desgaste prematuro de las celdas y asegurar la longevidad del dispositivo.

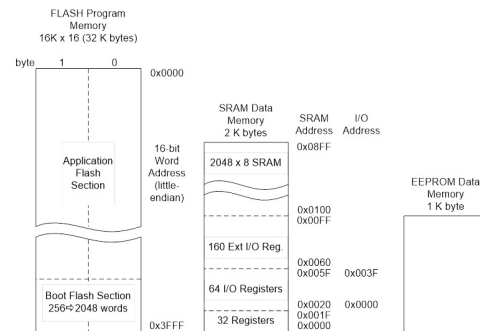


Figura 8. Estructura jerárquica de memoria del microcontrolador ATmega328P, que incluye los espacios de memoria *Flash*, *SRAM* y *EEPROM*. Cada uno cumple una función específica: la memoria *Flash* almacena el programa, la *SRAM* gestiona las variables temporales en ejecución y la *EEPROM* conserva datos no volátiles. Fuente: <https://www.arxterra.com/3-addressing-modes/>.

La EEPROM se utiliza comúnmente en sistemas embebidos para conservar información que debe persistir entre encendidos, como parámetros de calibración, configuraciones del sistema o datos de usuario. Su manejo puede realizarse mediante instrucciones específicas del microcontrolador o por medio de librerías de más alto nivel que abstraen las operaciones de lectura y escritura, garantizando un acceso controlado y confiable a la memoria no volátil [8].

II-F. Planificación temporal y multitarea cooperativa

En los sistemas embebidos, la ejecución de múltiples procesos requiere una estrategia de **planificación temporal** que permita coordinar distintas tareas sin provocar bloqueos ni retardos innecesarios [3]. Una de las técnicas más empleadas en microcontroladores de propósito general es la **multitarea cooperativa**, en la cual las tareas se ejecutan de manera secuencial dentro del ciclo principal (*main loop*) y ceden voluntariamente el control al resto una vez completada su función [7].

A diferencia de los sistemas con planificación basada en interrupciones o sistemas operativos en tiempo real (RTOS), la multitarea cooperativa no requiere mecanismos de cambio de contexto ni gestión de múltiples pilas, lo que la hace especialmente adecuada para dispositivos con recursos limitados. En este modelo, cada proceso es responsable de

ejecutarse rápidamente y permitir la continuidad del flujo general del programa, garantizando un funcionamiento fluido y determinista [6].

Esta técnica permite estructurar el programa en módulos independientes que se ejecutan de forma periódica o condicional, favoreciendo un diseño más claro y eficiente. Su simplicidad y bajo consumo de recursos la convierten en una herramienta fundamental en el desarrollo de sistemas embebidos donde la sincronización temporal es crítica y la capacidad de procesamiento es reducida.

III. METODOLOGÍA

III-A. Materiales y Herramientas

- Microcontrolador ATmega328P (Arduino Uno)
- Sensor LDR y LED RGB
- Servomotor SG90
- Teclado matricial 4x4
- Pantalla LCD 16x2 con interfaz I2C
- Buzzers piezoeléctricos (activo y pasivo)
- Tira LED WS2812
- Resistencias, cables, protoboard
- Plotter
- Software: Microchip Studio, Proteus / PicSimLab, Python (para generación de secuencias del plotter)

III-B. Procedimiento general

Se dividió el sistema en cuatro módulos independientes (Plotter, Colores, Piano y Cerradura) y se abarcó cada consigna de manera individual.

III-B1. Plotter:

Movimiento del plotter: — El sistema de trazado está controlado por un PLC, encargado de accionar los motores responsables del desplazamiento en los ejes X e Y. El microcontrolador ATmega328P, mediante comunicación serial, transmite al PLC las instrucciones que determinan el sentido y la magnitud de los movimientos. Según la secuencia enviada, el conjunto de actuadores traza distintas figuras geométricas o patrones definidos en memoria.

Comunicación vía USART: — La comunicación entre el microcontrolador y el PLC se realiza mediante el protocolo USART, configurado a 9600 bps y formato 8N1. A través de esta interfaz se transmiten las órdenes de movimiento y los estados de control necesarios para el dibujo. En figuras simples, como cruces o triángulos, los movimientos se generan mediante señales de duración controlada, mientras que en figuras más complejas se utilizan secuencias discretas equivalentes a píxeles. Esta metodología permite equilibrar precisión y tiempo de ejecución según la figura a trazar.

Procesamiento de datos en Python: — Con el fin de automatizar la generación de trayectorias, se desarrolló un script en *Python* que emplea librerías de procesamiento de imágenes para obtener el contorno de una figura y convertirlo en coordenadas discretas de desplazamiento. Estos datos se almacenan en un archivo `.txt`, que luego se carga en la memoria del ATmega328P como una *Look-Up Table*

(LUT), permitiendo al microcontrolador ejecutar el trazado sin procesamiento adicional durante la ejecución.

Dibujo del círculo mediante funciones trigonométricas: — Para el caso del círculo, se implementó un algoritmo alternativo basado en funciones trigonométricas. Utilizando las librerías matemáticas del lenguaje C, se calcularon las coordenadas de desplazamiento para cada incremento angular dentro del rango de 0° a 90° . De este modo, el sistema determina los desplazamientos exactos en los ejes X e Y, generando un cuarto de circunferencia mediante pequeños escalones. Repitiendo el procedimiento en los cuatro cuadrantes y combinando los movimientos UP, DOWN, LEFT y RIGHT, se obtiene la trayectoria completa del círculo.

III-B2. Colores:

Descomposición de colores: — El proceso de reconocimiento de colores se fundamenta en la descomposición de cada color en sus componentes primarias dentro del modelo RGB (*Red, Green, Blue*). Un color puede representarse como la combinación ponderada de estas tres intensidades, lo que permite su descripción en un espacio tridimensional.

Sin embargo, el sensor fotoresistivo (LDR, *Light Dependent Resistor*) utilizado en el sistema no distingue de forma individual las componentes cromáticas del espectro visible; únicamente mide la intensidad total de luz reflejada sobre su superficie. Si la iluminación se realiza con luz blanca, el sensor solo entrega una lectura proporcional a la cantidad total de luz reflejada, sin discriminar su composición espectral. Este fenómeno equivale a una percepción en escala de grises, dificultando la identificación precisa de colores.

Para resolver esta limitación, se empleó un diodo emisor de luz RGB como fuente de iluminación controlada. Al iluminar secuencialmente la superficie con luz roja, verde y azul, el sistema obtiene tres mediciones independientes mediante el LDR. Dichos valores, convertidos por el módulo ADC (*Analog-to-Digital Converter*) del microcontrolador ATmega328P, representan las coordenadas (*R, G, B*) de un punto dentro de un espacio de color tridimensional (véase anexo B2b).

Calibración y reconocimiento: — Dado que tanto la respuesta espectral del LDR como la emisión de los LED presentan variaciones no lineales, los valores obtenidos no son directamente proporcionales a las componentes RGB teóricas. No obstante, se implementa un proceso de calibración inicial para contrarrestar estos efectos, donde se miden manualmente los colores de referencia (rojo, verde, azul claro, violeta, morado, amarillo y blanco) y se almacenan sus valores de respuesta RGB de conversión analógica-digital.

Cada color de referencia calibrado se modela como un vector fijo en dicho espacio. Para identificar un color, se calcula la distancia euclidiana entre el vector de medición y cada uno de los vectores de referencia almacenados:

$$D = \sqrt{(R_m - R_i)^2 + (G_m - G_i)^2 + (B_m - B_i)^2} \quad (1)$$

donde (R_m, G_m, B_m) representan los valores medidos por el sensor y (R_i, G_i, B_i) corresponden a los valores calibrados de cada color de la hoja de referencia. El color identificado será aquel cuya distancia D sea mínima (véase anexo B2f).

Servomotor y tira LED: — El servomotor se controla mediante el *Timer1* configurado en modo *Fast PWM* a 50 Hz, generando pulsos de entre 1 y 2 ms para posicionar el eje según el color identificado. Cada ángulo se calcula a partir del valor del registro *OCR1A*, que define el tiempo en alto del ciclo PWM (véase anexo B2e).

La visualización del color se realiza con una tira de LED WS2812, donde cada diodo recibe datos digitales codificados en tres bytes (RGB). Para cumplir con los tiempos estrictos del protocolo, se emplearon instrucciones `asm volatile` que garantizan una sincronización precisa en la transmisión de pulsos. Así, al identificar un color, el sistema replica su tonalidad en la tira LED y mueve el servomotor al ángulo correspondiente, integrando una respuesta visual y mecánica simultánea (véase anexo B2d).

USART: — A través de USART se comunican los valores del sensor y el color detectado hacia una interfaz serial. Se utiliza un buffer circular para transmisión continua sin bloqueo, y la función auxiliar *UTOA* convierte los datos numéricos del ADC en texto ASCII antes de su envío. De esta forma, el sistema comunica en tiempo real los valores (R, G, B) medidos, el color identificado y el ángulo del servomotor, facilitando el monitoreo y la calibración del sistema (véase anexo B2g).

III-B3. Piano:

Notas musicales: — El sonido es una onda mecánica que se propaga a través de un medio elástico, producto de variaciones periódicas de presión. Estas ondas pueden clasificarse según su forma en senoidales, cuadradas, triangulares o diente de sierra, dependiendo del patrón temporal de oscilación. En los sistemas digitales, las señales más sencillas de generar son las ondas cuadradas, donde el voltaje alterna entre dos niveles definidos, representando los estados lógicos alto y bajo del microcontrolador.

Para producir sonido de manera electrónica, se emplean dispositivos piezoeléctricos denominados *buzzers*. Estos transductores convierten la energía eléctrica en vibraciones mecánicas audibles. Cuando el microcontrolador aplica una señal cuadrada a la entrada del buzzer, el material piezoeléctrico se deforma y contrae periódicamente, generando un sonido cuya frecuencia está directamente relacionada con la frecuencia de la señal de entrada.

En el microcontrolador ATmega328P, esta señal se genera mediante el uso de los temporizadores internos (*timers*),

configurados para producir una onda cuadrada en un pin de salida. Al modificar la frecuencia de conmutación, es posible variar el tono percibido, ya que la frecuencia del sonido determina su altura musical. De esta manera, cada nota puede asociarse a una frecuencia específica según la escala. Por ejemplo, la nota *La4* corresponde a una frecuencia de 440 Hz, mientras que *Do4* equivale aproximadamente a 262 Hz (véase anexo B3b).

Generación de frecuencias: — Para reproducir una nota musical, el microcontrolador debe generar una señal cuadrada con una frecuencia específica, determinada por el valor de comparación del temporizador. En el ATmega328P, dicha frecuencia depende de la relación entre la frecuencia del reloj del sistema (F_{CPU}), el valor de comparación *OCRx*, y el prescaler aplicado al temporizador. Este último actúa como un divisor de frecuencia, reduciendo la velocidad a la que el contador incrementa.

Elegir el prescaler adecuado resulta fundamental para asegurar que el valor de *OCRx* se mantenga dentro del rango permitido por el temporizador (por ejemplo, 8 bits en el *Timer0*). Si la frecuencia deseada es alta, se requiere un prescaler pequeño para mantener precisión; si es baja, un prescaler grande evita desbordamientos y permite generar tonos audibles.

En la práctica, el programa selecciona dinámicamente el prescaler que produce un valor de *OCR0A* válido para la nota solicitada (véase anexo B3c). De esta forma, es posible generar un amplio rango de frecuencias musicales con un solo temporizador, manteniendo una relación estable entre tono y duración.

Reproducir música: — Una nota musical puede representarse computacionalmente mediante tres parámetros fundamentales: la frecuencia de oscilación, la duración temporal durante la cual se mantiene activa y la duración inactiva o en silencio. Cuando el buzzer emite una secuencia ordenada de notas, cada una con su frecuencia, tiempo de activación, y silencio, se obtiene una melodía. En términos programáticos, una melodía se define como un conjunto de estructuras de datos que contienen pares (*frecuencia*, *tiempo_on*, *tiempo_off*), los cuales son procesados secuencialmente para generar la música deseada.

Para crear pistas de canciones se utilizó la herramienta *MIDI to Arduino Converter* [9], la cual permite convertir pistas MIDI al formato anteriormente mencionado, y almacenarlas dentro de la memoria del microcontrolador.

La reproducción simultánea de varios instrumentos en un entorno digital requiere el manejo de múltiples secuencias de notas en paralelo. Para lograrlo sin recurrir al uso de ciclos de espera activos (*polling*), se utilizan interrupciones asociadas a los temporizadores. De este modo, el microcontrolador puede controlar la duración y el inicio de cada nota de forma independiente y precisa, optimizando el uso del procesador

y permitiendo la ejecución de tareas concurrentes, como la lectura de entradas o la comunicación serial, mientras la música continúa sonando de manera autónoma (véase anexo B3d).⁹

Modos de funcionamiento y USART: — Para implementar el modo piano se decidió utilizar 8 pulsadores correspondientes a las 8 principales notas musicales. Cada uno de los pulsadores fue conectado a un pin del puerto D del microcontrolador, configurando los pines como entradas y pull-ups internas. Utilizando interrupciones externas por cambio de pin (PCINT) y lógica programática para debouncing, se implementó la funcionalidad de que mientras el pulsador está presionado, la nota sigue sonando.

Finalmente, a este sistema se integra funcionalidad USART para mostrar un menú al usuario por consola y permitir comandar el funcionamiento del sistema de manera sencilla. El usuario es presentado con 3 opciones: [1]: Reproducir canción 1, [2]: Reproducir canción 2, [P]: Cambiar a modo piano. Mediante variables de control se deshabilita la funcionalidad de piano durante la reproducción de canciones, y vice versa, al cambiar al modo piano se detiene y reinician las pistas que se estaban reproduciendo (véase anexo B3e).

En síntesis, el sistema desarrollado utiliza un buzzer piezoeléctrico controlado por los temporizadores del ATmega328P para reproducir notas musicales definidas por su frecuencia y duración. A través de la programación de secuencias almacenadas en memoria, es posible interpretar melodías completas y gestionar la reproducción de distintas canciones sin intervención del usuario durante la ejecución. Utilizando USART se puede comandar al programa para reproducir diferentes pistas guardadas o cambiar a modo piano donde 8 pulsadores reproducen las 8 notas principales.

III-B4. Cerradura:

Idea general: — El sistema de cerradura electrónica desarrollado tiene como objetivo controlar el acceso mediante una contraseña numérica almacenada en memoria no volátil. Inicialmente, el dispositivo se encuentra en estado de *candado cerrado*. Cuando el usuario ingresa la contraseña correcta, el sistema cambia al estado de *candado abierto*, permitiendo el acceso. En caso de introducir una contraseña incorrecta tres veces consecutivas, se activa una alarma acústica a través de un buzzer, indicando un intento de acceso no autorizado.

El sistema también permite modificar la contraseña almacenada. Para ello, el usuario debe ingresar primero la contraseña actual y, tras su validación, definir una nueva contraseña de entre cuatro y seis dígitos. Esta nueva clave se almacena permanentemente en la memoria EEPROM del microcontrolador, garantizando su persistencia incluso después de un apagado o reinicio del sistema.

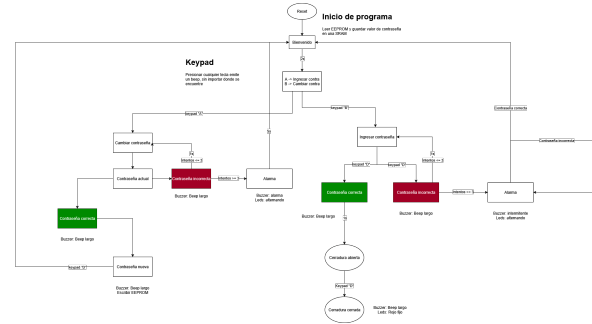


Figura 9. Diagrama de flujo de programa de cerradura. Fuente: elaboración propia

Interfaz de usuario: — La cerradura dispone de una interfaz de usuario compuesta por una pantalla LCD de 16x2, un teclado matricial 4x4 y tres indicadores luminosos (LED verde, LED rojo y alarma sonora). En este contexto, la interfaz constituye el medio de comunicación entre el usuario y el sistema, mostrando mensajes informativos y recibiendo acciones por medio del teclado. Cada interacción del usuario genera una respuesta visual o acústica distinta, representando así un flujo de diálogo entre ambos.

El sistema se diseñó siguiendo una lógica de *máquina de estados finitos*, en la que cada modo de operación (menú principal, ingreso de contraseña, cambio de clave, acceso autorizado, alarma, etc.) representa un estado. Las transiciones entre estados se producen en función de las acciones del usuario y las condiciones del sistema. Este enfoque facilita la gestión de comportamientos complejos, simplifica el control del flujo de ejecución y mejora la legibilidad del código (véase anexo B4b).

Teclado matricial: — El ingreso de datos se realiza mediante un teclado matricial 4x4 que combina filas y columnas, optimizando el uso de pines del microcontrolador. Las filas se configuran como salidas y las columnas como entradas con resistencias de *pull-up* activadas. El proceso de lectura consiste en activar una fila a nivel bajo (0) mientras las demás permanecen en alto (1); si alguna tecla de esa fila se encuentra presionada, la columna correspondiente cambia a nivel bajo, permitiendo identificar la intersección entre fila y columna (véase anexo B4c).

Cada tecla se asocia a un carácter según su posición dentro de la matriz, lo que permite retornar el valor numérico o simbólico correspondiente. Para evitar falsas detecciones debido al rebote mecánico de los contactos, se aplica una rutina de *debouncing* por software basada en pequeños retardos temporizados.

Planificador de tareas: — Durante el funcionamiento, el sistema debe ejecutar múltiples tareas de forma paralela: reproducción de sonidos, manejo de alarmas, parpadeo de LEDs, lectura del teclado y actualización de la pantalla LCD. Sin embargo, el ATmega328P dispone de un número

limitado de temporizadores, por lo que no es posible asignar un temporizador independiente a cada tarea.

Para resolver esta limitación, se implementó un esquema de ejecución basado en un *planificador de tareas* o *task scheduler* de propósito general. Se utiliza un solo temporizador configurado para generar interrupciones periódicas, incrementando un contador global de 32 bits que actúa como referencia temporal (en milisegundos). Cada tarea compara el tiempo actual con el instante programado de su próxima ejecución, y si se cumple el intervalo, se ejecuta la acción correspondiente. De este modo, múltiples eventos temporizados pueden coexistir sin emplear retardos bloqueantes ni técnicas de *polling* (véase anexo B4d).

Almacenamiento en EEPROM: — Para asegurar que la contraseña permanezca almacenada tras un apagado, se utiliza la memoria EEPROM interna del ATmega328P. Esta memoria no volátil permite conservar los datos durante décadas sin alimentación eléctrica, aunque posee un número limitado de ciclos de escritura (aproximadamente 10^5 operaciones por celda). Por ello, el sistema únicamente escribe en EEPROM cuando el usuario confirma un cambio de contraseña, minimizando el desgaste de la memoria.

La lectura y escritura se realizan mediante las funciones de la biblioteca estándar `avr/eeprom.h`, que permiten transferir cadenas de caracteres directamente desde y hacia direcciones específicas de la EEPROM. Así, el sistema recupera la contraseña almacenada al inicio del programa y la compara con la ingresada por el usuario durante la operación normal (véase anexo B4e).

En conjunto, la cerradura electrónica combina la interacción mediante teclado y pantalla LCD, la gestión de tareas en tiempo real y el almacenamiento persistente de datos en memoria no volátil EEPROM. El uso de una máquina de estados y un planificador temporal basado en un único temporizador permite controlar de manera eficiente múltiples procesos simultáneos sin bloquear la ejecución principal del programa.

IV. RESULTADOS

En esta sección se presentan los resultados obtenidos durante la implementación y prueba de los distintos módulos desarrollados: el sistema de reconocimiento de colores, el piano electrónico, la cerradura con contraseña y el control del plotter. Exceptuando este último, cada módulo fue verificado individualmente en simulación y en montaje físico, registrando su funcionamiento y las principales observaciones.

IV-A. Resultados por módulo

IV-A1. Plotter: —

Comunicación USART: Para configurar USART se uso una configuración de buffer circular tanto para la recepción como para el envío de datos al atmega (véase

anexo B1a). El menu de opciones para dibujar figuras es el que se puede ver en la figura 10.

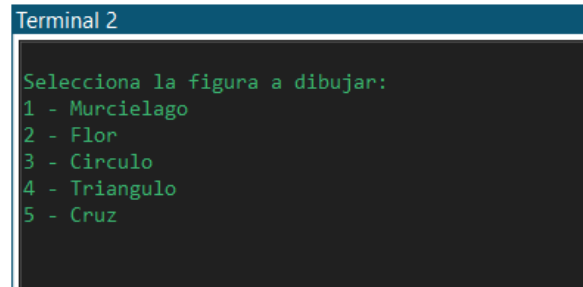


Figura 10. Menu de seleccion de figura.

Declaracion de variables globales: Para controlar el movimiento del plotter de manera adecuada, es necesario considerar que las señales de activación de los motores se determinan según la tabla 11. Con el fin de simplificar el código y su implementación, se emplearon variables globales, presentadas en la tabla I, las cuales representan los diferentes estados de dirección y control. Para garantizar un desplazamiento correcto, cada vez que una de estas variables se asigna a los pines de salida del puerto D, se aplica una operación lógica AND. Esto permite aislar los bits correspondientes al movimiento del eje, evitando interferir con las señales destinadas al accionamiento del pistón que eleva o desciende el lápiz, responsable de establecer o interrumpir el contacto con la hoja. Se puede ver en anexo B1b.

Pin Digital	Conexión
D2	Bajar solenoide X0
D3	Subir solenoide X1
D4	Movimiento hacia abajo X5
D5	Movimiento hacia arriba X6
D6	Movimiento hacia la izquierda X7
D7	Movimiento hacia la derecha X10

Figura 11. Menu de seleccion de figura.

Cuadro I
CÓDIGOS DE DIRECCIÓN Y CONTROL UTILIZADOS EN EL FIRMWARE DEL PLOTTER

Código (macro)	Descripción	Valor
SOLENOID_DOWN	Bajar solenoide (pluma en contacto)	1
SOLENOID_UP	Subir solenoide (pluma levantada)	2
DOWN	Movimiento hacia abajo	3
UP	Movimiento hacia arriba	4
RIGHT	Movimiento a la derecha	5
LEFT	Movimiento a la izquierda	6
STOP	Fin de secuencia / detener	7
UP_RIGHT	Arriba + derecha	8
UP_LEFT	Arriba + izquierda	9
DOWN_RIGHT	Abajo + derecha	10
DOWN_LEFT	Abajo + izquierda	11

Para simplemente controlar el selenoide a gusto, en la etapa de pruebas, tambien se implemento un modo de movimiento libre. Donde el mismo usuario es el que elige como mover el lapiz en todo momento. Vease anexo B1c.

IV-A1a. Implementacion del círculo mediante trigonometria: Para generar el círculo mediante cálculo inmediato se consideró la relación trigonométrica de un triángulo rectángulo. En este caso, la hipotenusa representa el radio del círculo, mientras que el cateto opuesto corresponde a la altura de cada fragmento individual del trazo y el cateto adyacente a la distancia en el eje x asociada a dicho fragmento. De esta forma, aplicando las funciones seno y coseno se obtiene la posición incremental del punto en cada eje, permitiendo describir el arco del círculo mediante pequeños desplazamientos sucesivos del *plotter*. Siendo modificado unicamente, el tiempo de desplazamiento para subir y bajar el trazo. Es decir se emplean escalones que se van adaptando al ángulo del fragmento trazado.

Para realizar el calculo se empleo la funcion **void precompute semicirculo arrays 90°**, que se encarga de tomar los angulos comprendidos entre 0 y 90 grados, para pasarlos a radianes cada uno de forma individual. Aplicando seno y coseno para cada angulo, a su vez se le agrega una verificación en caso de que el valor devuelto sea mayor a 1 o menor a 0 (Ya que pierde la utilidad y afecta al trazado, llevandolo fuera del círculo). Y así mutiplicar a **I ms** por el valor comprendido entre 0 y 1. Para luego guardar dicho valor en los arreglos **TR** y **TU**(vease anexo B1d). De esta forma se varia el tamaño del círculo variando el valor de **I ms**. Una representacion grafica que ayuda a entender la implementacion, se puede ver en la figura 12.

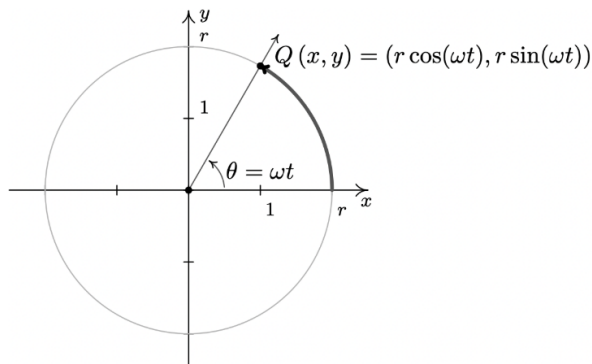


Figura 12. Demostracion

El trazado del círculo se inicia en la función **Hacer círculo**, la cual completa los 360° utilizando los mismos valores de coordenadas para los cuatro cuadrantes. La diferencia entre cada uno radica únicamente en el tipo de movimiento aplicado. En el primer cuadrante, el desplazamiento se realiza primero hacia la derecha y luego hacia arriba; en el segundo, hacia la derecha y posteriormente hacia abajo; en el tercero, primero hacia la izquierda y luego hacia abajo; y en el cuarto, el movimiento es ascendente seguido de un desplazamiento hacia la izquierda. De esta forma, reutilizando los mismos cálculos trigonométricos y variando

únicamente el orden y dirección de los movimientos (así como los tiempos de ejecución) se logra completar el trazado continuo de todo el círculo con una implementación.

IV-A1b. Tomar trazado con Python: Dado que la implementación se realizó en Google Colab, se incorporaron librerías específicas para acceder a imágenes almacenadas en Google Drive. El flujo de procesamiento inicia cargando la imagen desde Drive y convirtiéndola, mediante *Pillow* (PIL), en un objeto manipulable con metodos que señalan color, ángulo, etc; posteriormente, se transforma a una matriz **NumPy** para operar píxel a píxel, donde cada elemento representa un color en el espacio **RGB**.

A continuación, la imagen se convierte a escala de grises con **cv2.cvtColor** y se aplica un suavizado con **cv2.GaussianBlur** para reducir ruido y facilitar la detección de bordes. Luego, se ejecuta el detector de bordes (**cv2.Canny**) y se obtienen los contornos mediante **cv2.findContours**; para visualización, se dibuja el contorno externo sobre la imagen original. El código completo de este procesamiento se presenta en el anexo B1e. En este punto, se recorren los contornos extraídos de la imagen y se traducen a una secuencia de comandos adaptados a las variables globales del *plotter*. Primero se verifica que exista al menos un contorno y, para cada uno, se descartan los casos degenerados con menos de dos puntos (no permiten definir un desplazamiento). Acto seguido, se inserta el comando **SOLENOID_DOWN** para iniciar el trazado con el lápiz en contacto.

Luego, el algoritmo itera punto a punto sobre cada contorno, tomando pares consecutivos (x_1, y_1) y (x_2, y_2) . A partir de las diferencias dy y dx se determina una etiqueta de dirección entre ocho posibles (cardinales y diagonales): **RIGHT**, **LEFT**, **UP**, **DOWN**, **UPRIGHT**, **UPLEFT**, **DOWNRIGHT**, **DOWNLEFT**. Si $\Delta x = \Delta y = 0$, se etiqueta como **STAY**. Esta clasificación se basa en el signo y la nulidad relativa de Δx y Δy . Es importante recordar que, en el sistema de coordenadas de imagen, el eje y crece hacia abajo; por ello, $\Delta y > 0$ se interpreta como *descenso* (**DOWN**) y $\Delta y < 0$ como *ascenso* (**UP**). Cada etiqueta resultante se agrega secuencialmente a la lista de movimientos.

Una vez procesados todos los segmentos del contorno, se añade **SOLENOID_UP** para levantar el lápiz y finalizar el trazo de esa forma. El resultado es una cadena de comandos *no bloqueante* que codifica el recorrido del contorno con el solenoide coordinado.

La secuencia de movimientos se almacena como cadena de comandos en un archivo .txt para su posterior uso en el firmware del **plotter**. Finalmente, con el fin de facilitar la carga manual en la LUT, los elementos se formatean en renglones de 15 entradas, separados por comas. La implementación de este formateo se muestra en el anexo B1f, y un ejemplo del contorno extraído puede observarse en la figura 13.



Figura 13. Imagen contorno

IV-A1c. Ejecucion del trazado: Para ejecutar el trazado en el plotter se empleo la funcion **Ejecutar trazado**. Que toma el arreglo declarado en la LUT para la figura correspondiente. Y evalua cada elemento hasta encontrar un STOP, en cada elemento evaluando va seteando las salidas del puerto D. Para saber que tiene que hacer se utiliza un simple Switch case. Se puede ver la funcion en el anexo B1g.

Las figuras representadas mediante este metodo, fueron el Murcielago, La flor y el circulo. En las figuras 14, 15 y 16 se puede ver cada una correspondiente.



Figura 14. Imagen Murcielago



Figura 15. Imagen Flor

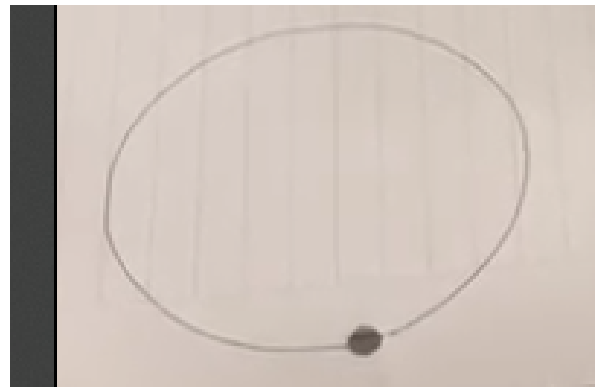


Figura 16. Imagen Circulo

IV-A1d. Triangulo: Para el triángulo se reutilizaron las funciones de movimiento previamente descritas, parametrizando el tiempo de activación para cada tramo. La figura corresponde a un triángulo isósceles: se asignó un tiempo específico para la línea de la base y otro para cada uno de los dos lados restantes. En la práctica, se observó un ligero retardo de respuesta durante el trazado de la base que tendía a desalinear el vértice de cierre; para compensarlo, se incrementó levemente el tiempo de la base respecto del de los catetos, garantizando el cierre geométrico del contorno.

La imagen del triángulo obtenido se presenta en la figura 17, y la función correspondiente se incluye en el anexo B1h.

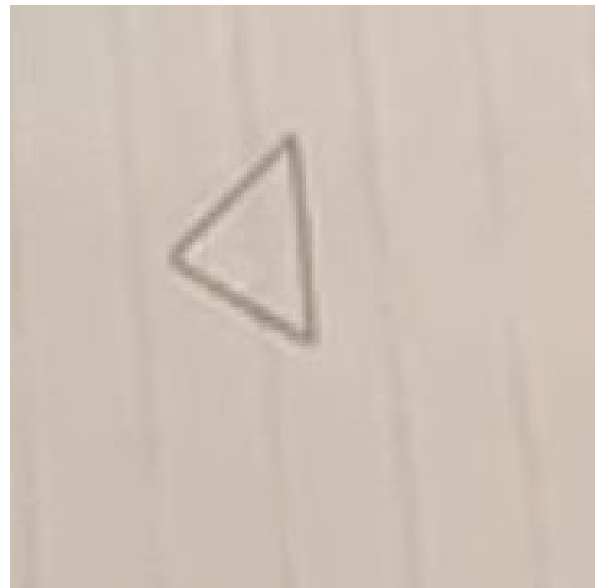


Figura 17. Imagen Triangulo

IV-A1e. Cruz: Para la cruz se empleo el codigo que se puede ver en el anexo B1i. Se utilizaron los movimientos en diagonal del plotter, moviendo para una diagonal hasta llegar al final. Volver al medio de esa linea, y ahi ejecutar la segunda diagonal.

El video de todas las figuras juntas se puede encontrar en el drive del laboratorio (vease anexo ??).

IV-A2. Colores: —

Simulación: —

La simulación en Proteus permitió verificar el funcionamiento lógico del sistema antes de su implementación física. En el entorno virtual, se replicó el comportamiento del LDR mediante una fuente de voltaje variable que simulaba distintos niveles de iluminación. Al variar dicho valor, se observaron cambios coherentes en la salida del ADC, en la posición del servomotor y en el color representado por la tira de LEDs. Esto confirmó la correcta interacción entre los módulos de lectura, procesamiento y visualización del sistema. La simulación también permitió ajustar los tiempos de respuesta y verificar la estabilidad del ciclo de lectura sin la influencia de ruido externo.

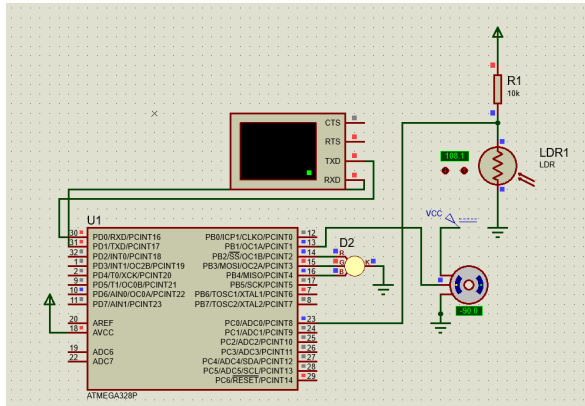


Figura 18. Simulación en Proteus del sistema de detección de color. Se observa la variación de la señal del LDR y la correspondiente respuesta del servomotor y de la tira LED. Fuente: elaboración propia.

Servomotor y calibración: —

Durante el proceso de calibración se determinó que la estabilidad del sistema dependía en gran medida de la fuente de alimentación utilizada. Cuando el servomotor era alimentado únicamente desde el regulador de 5 V del propio Arduino, se observaban fluctuaciones de tensión en los momentos de arranque debido a los picos de corriente característicos del motor. Esto provocaba lecturas erróneas del sensor LDR y una respuesta inestable en la identificación del color, generando un bucle de retroalimentación entre la medición y el movimiento del servo. La situación se resolvió alimentando el servomotor desde una fuente externa regulada, lo que permitió aislar las variaciones de corriente y obtener una lectura más consistente de los valores de referencia.

Cuadro II
VALORES DE REFERENCIA RGB PARA CADA COLOR CALIBRADO.

Color	R	G	B
Morado	216	157	274
Rojo	206	149	262
Amarillo	196	99	105
Verde	272	192	152
Azul Claro	151	153	122
Violeta	253	275	338
Blanco	110	93	91

Reconocimiento decolores y manejo de la tira LED: —

Además, se observó que la cantidad de luz ambiental influía directamente en la precisión de la lectura del sensor. La misma tira de LEDs WS2812 utilizada para la representación del color introducía una leve interferencia óptica sobre la LDR cuando se encontraba demasiado cerca, provocando pequeñas desviaciones en las mediciones. Para mitigar este efecto, se implementó un apantallamiento parcial que bloqueó la incidencia directa de la luz de la tira sobre el sensor, manteniendo al mismo tiempo una iluminación uniforme sobre la superficie de calibración. Con estas modificaciones, el sistema logró una detección estable y una correspondencia confiable entre los valores ADC medidos y los colores calibrados.

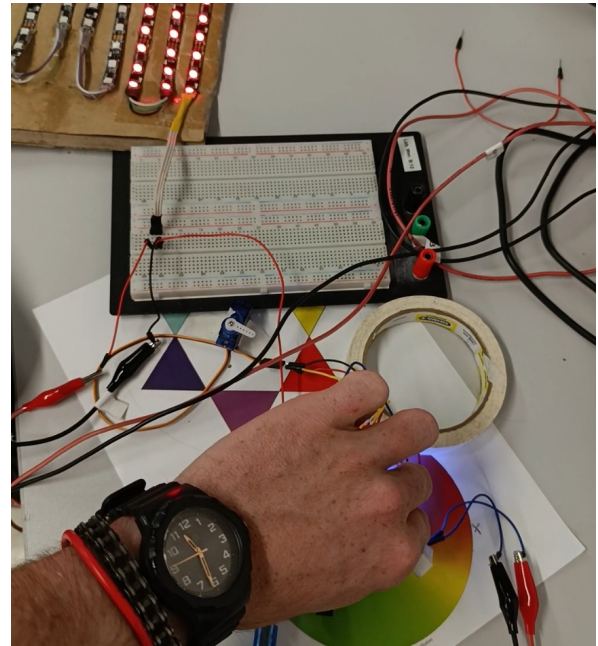


Figura 19. Identificación de color efectiva. Fuente: elaboración propia

Comunicación USART: —

Finalmente, durante la implementación de la comunicación serial mediante USART se detectó una ralentización perceptible en el ciclo principal del programa. Esto se debía al tiempo requerido para transmitir cada paquete de datos hacia la terminal, especialmente cuando se imprimían valores en cada iteración del bucle. Para evitar sobrecargas en el búfer de transmisión (*overflow*) y asegurar una comunicación confiable, fue necesario introducir retardos (*delay*) calibrados entre envíos sucesivos, limitando así la frecuencia de actualización sin comprometer la precisión del monitoreo. La Figura 20 muestra la salida obtenida en la terminal serial, donde se visualizan el valor analógico medido, el color identificado, el valor de referencia y la desviación asociada.

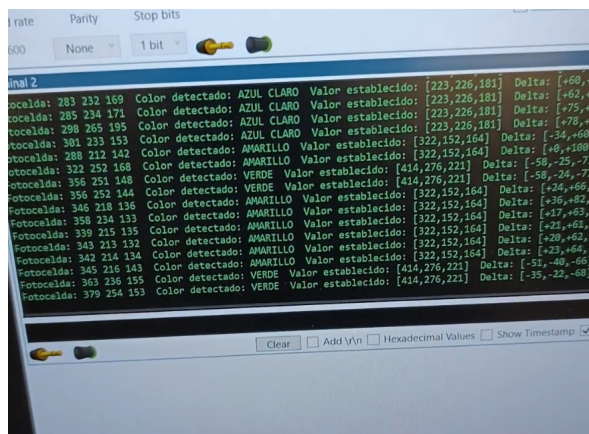


Figura 20. Identificación de color efectiva mediante comunicación serial a través de la terminal. Se observa la lectura del valor ADC, el color detectado, el valor establecido y la desviación del mismo. Fuente: elaboración propia.

IV-A3. Piano: —

Simulación: —

La simulación en **Proteus** permitió verificar el funcionamiento general del sistema de piano electrónico antes del montaje físico. Se observó que al accionar los pulsadores se generaban las notas esperadas y que las interrupciones por cambio de pin respondían de forma estable, reproduciendo correctamente los tonos tanto en modo manual como en modo automático. El uso del microcontrolador **ATmega328P** de forma independiente, en lugar del módulo **Arduino Uno**, corrigió los problemas iniciales con las interrupciones **PCINT**, logrando una respuesta fluida en la reproducción de las melodías.

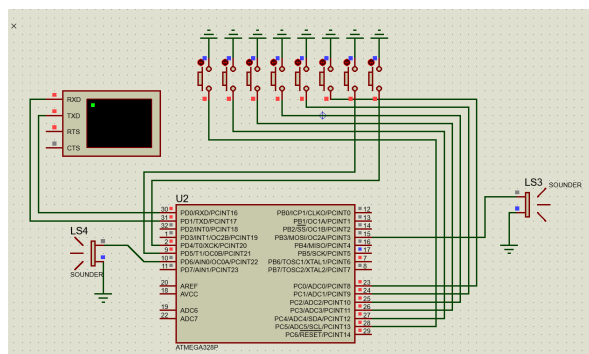


Figura 21. Simulación en Proteus del sistema de piano electrónico. Se observa la activación de los pulsadores, la generación de tonos en el buzzer y la respuesta del microcontrolador ATmega328P configurado con interrupciones **PCINT**. Fuente: elaboración propia.

Generación de frecuencias: —

Durante las pruebas físicas, el sistema generó correctamente las frecuencias correspondientes a cada nota musical. Las señales cuadradas producidas fueron estables y sin distorsiones audibles, manteniendo una relación precisa entre la frecuencia programada y el tono percibido. El buzzer respondió con buena fidelidad en todo el rango de notas implementado, sin interferencias ni retardos perceptibles al presionar las teclas.

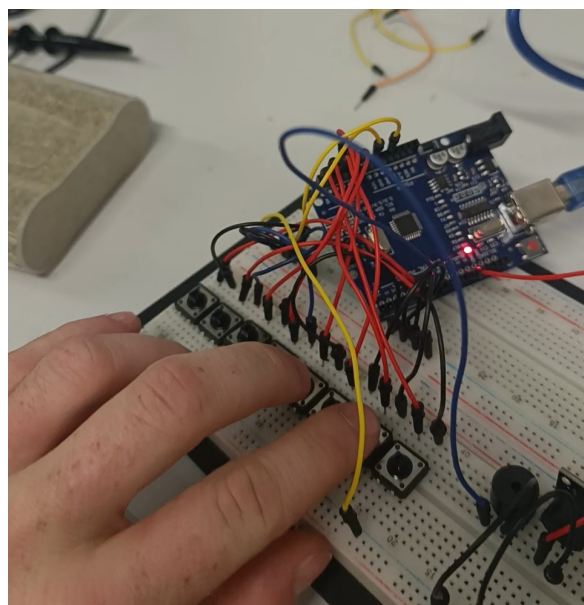


Figura 22. Montaje del circuito correspondiente al módulo **Piano**, compuesto por el microcontrolador ATmega328P, el buzzer piezoeléctrico, los pulsadores de entrada y los componentes de conexión. Fuente: elaboración propia.

Reproducción de música: —

El sistema reprodujo correctamente las melodías preprogramadas, manteniendo el ritmo y la secuencia esperada de las notas. Se detectaron leves desfases entre las pistas durante la ejecución, lo cuales se volvían más aparentes conforme avanzaba la reproducción. Para melodías con una sola pista no se detectaron desviaciones con las originales, lo que confirmó la estabilidad temporal del módulo. Las canciones fueron reproducidas con continuidad y volumen uniforme, evidenciando un control adecuado sobre la temporización y la generación de frecuencias.

Modos de funcionamiento y comunicación USART: —

Durante la verificación de los modos de operación, se comprobó que el sistema respondía correctamente tanto en el modo **manual** (piano) como en el modo **automático** (canciones). La interfaz serial **USART** permitió alternar entre los modos de forma confiable, así como iniciar y detener la reproducción desde la terminal. La comunicación se mantuvo estable a 9600 bps, sin pérdida de datos ni interferencia con la reproducción del sonido. El menú mostrado en la terminal facilitó el control y la supervisión del sistema durante las pruebas.

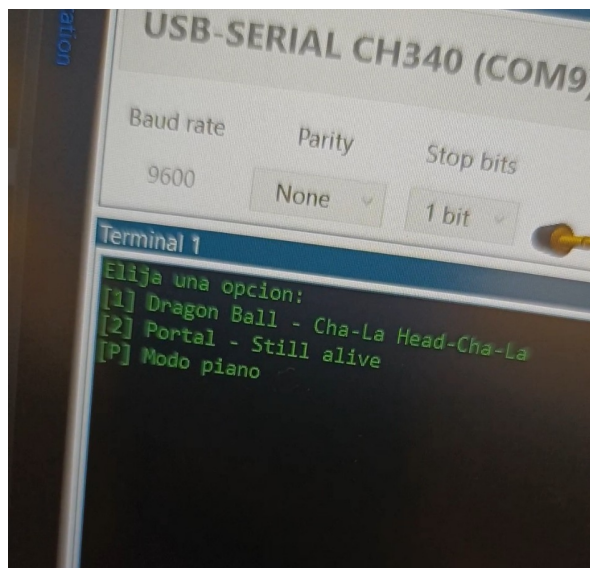


Figura 23. Interfaz de comunicación serial (USART) mostrando el menú de control del módulo *Piano*. Desde la terminal, el usuario puede seleccionar entre el modo manual por pulsadores o la reproducción automática de melodías predefinidas. Fuente: elaboración propia.

IV-A4. Cerradura: —

Simulación: —

La simulación del sistema se realizó inicialmente en **Proteus**, aunque se presentaron dificultades con la visualización en la pantalla LCD con interfaz I2C, la cual no respondió de forma adecuada a las señales del microcontrolador. Posteriormente, el circuito fue probado en **PicSimLab**, donde el sistema funcionó correctamente y permitió validar el funcionamiento del teclado matricial, la pantalla LCD, los LEDs indicadores y el buzzer. Durante las pruebas, el ingreso y verificación de contraseñas se comportaron de manera estable, y se observó la transición entre estados de bloqueo, desbloqueo y alarma. El almacenamiento en memoria EEPROM también pudo verificarse, conservando los datos tras los reinicios del sistema.

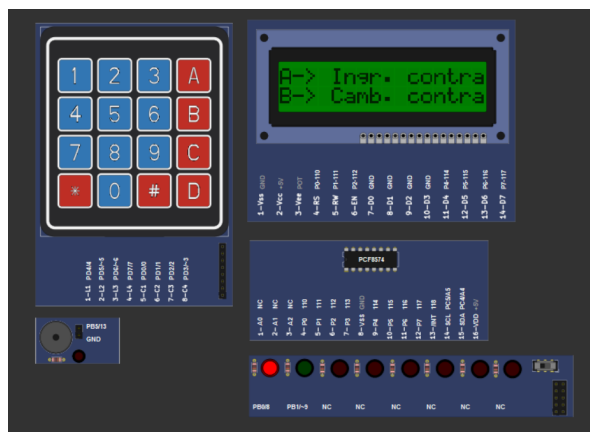


Figura 24. Simulación en PicSimLab del sistema de cerradura electrónica. Se observan los periféricos principales: teclado matricial, pantalla LCD, LEDs indicadores y buzzer, así como la correcta ejecución de las rutinas de interfaz y control de estado. Fuente: elaboración propia.

Teclado matricial: —

Durante la verificación del teclado matricial 4x4, se comprobó una lectura estable y sin errores de rebote, incluso con pulsaciones rápidas o repetitivas. El ingreso de la contraseña fue detectado con precisión, y el sistema respondió inmediatamente al comparar los dígitos con la clave almacenada. El comportamiento fue consistente tanto en simulación como en la prueba física, sin registros de lecturas duplicadas o fallas de detección.

Interfaz de usuario: —

En las pruebas de la interfaz LCD se confirmó que los mensajes se mostraban correctamente en cada etapa del proceso: inicio del sistema, solicitud de contraseña, acceso concedido o denegado y aviso de alarma. Tras tres intentos fallidos consecutivos, se activó la alarma sonora y el LED rojo de manera intermitente, mientras que el LED verde indicó exitosamente la apertura del sistema cuando la contraseña era correcta. La interacción visual y auditiva resultó clara y permitió identificar el estado del sistema en todo momento.

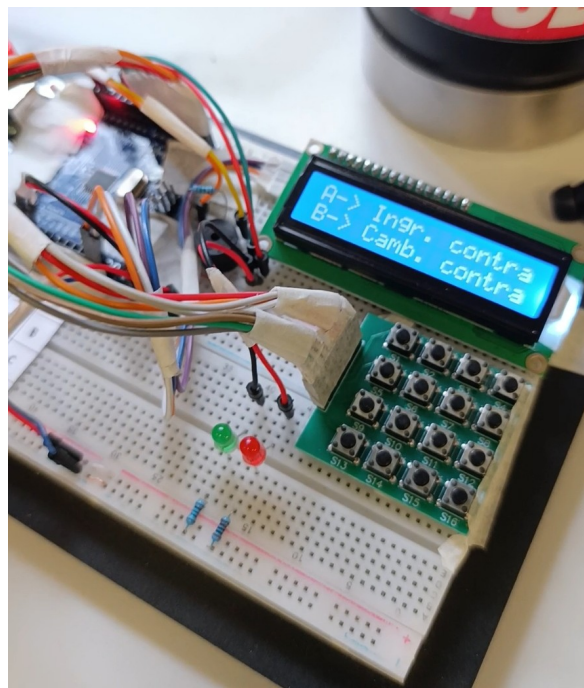


Figura 25. Montaje del circuito correspondiente al sistema de cerradura electrónica. Se observa la conexión del teclado matricial 4x4, la pantalla LCD 16x2 con interfaz I2C, los indicadores luminosos (LED rojo y verde), el buzzer de alarma y el microcontrolador ATmega328P. Fuente: elaboración propia.

Planificador de tareas: —

Durante las pruebas se verificó que el planificador de tareas permitió ejecutar de forma fluida las distintas funciones del sistema sin bloqueos perceptibles. El escaneo del teclado, la actualización del LCD, la supervisión de la alarma y el control de los LEDs se realizaron de manera sincronizada y estable, manteniendo una respuesta inmediata a las acciones del usuario. No se registraron conflictos temporales ni retrasos en la ejecución de las tareas programadas.

Almacenamiento en EEPROM: —

El sistema demostró un funcionamiento confiable del almacenamiento no volátil. La contraseña permaneció guardada tras múltiples reinicios, confirmando la correcta escritura y lectura en la memoria **EEPROM**. Durante las pruebas, se modificó la contraseña en varias ocasiones y los nuevos valores se conservaron correctamente, mostrando la robustez del procedimiento de guardado. En conjunto, el módulo cumplió con los objetivos previstos, garantizando una gestión segura y persistente de la información del usuario.

V. CONCLUSIONES

V-A. Plotter

El desarrollo del *plotter* permitió constatar que, para tareas de procesamiento de imagen o manejo de grandes volúmenes de datos, conviene delegar el cómputo a herramientas de alto nivel en un equipo externo. De esta forma, el microcontrolador queda focalizado en la ejecución determinista en tiempo real (activación de motores y solenoide), preservando la eficiencia de un lenguaje de bajo nivel y evitando sobrecargar recursos limitados (CPU, SRAM, temporizadores).

Esta separación de responsabilidades también visibilizó la necesidad de ajustes manuales y calibraciones en sistemas embebidos para corregir pequeñas irregularidades mecánicas o temporales (p.ej., compensaciones de tiempo en ciertos trazos). Se evaluó incluso un intento de generar el trazado por matriz directamente en C sobre el ATmega328P; no obstante, la complejidad frente al beneficio esperado y el límite temporal hicieron ver que no era buena alternativa.

Finalmente, resultó ilustrativo comprobar cómo, a partir de una secuencia de datos precomputada (LUT), un conjunto de instrucciones simples —subir/bajar y movimientos cardinales o diagonales— puede componer figuras complejas (murciélago, flor, círculo, triángulo) con un flujo robusto, reproducible y fácil de implementar. La utilización de librerías matemáticas también fue clave en esta implementación, ya que permiten manejar los datos de formas más complejas como se puede ver en el círculo por funciones trigonométricas.

Como mejora de la implementación final, se podría implementar un código en python que en vez de tomar solo el contorno externo, tome toda la figura, permitiendo ya realizar todo tipo de dibujos. Y a modo de más aprendizaje, realizar la implementación de un dibujo por matriz en C, aunque resulte más complejo. Solo para aprovechar el 100 por ciento del Atmega.

V-B. Colores

El desarrollo del sistema de detección de colores permitió comprender en profundidad el funcionamiento del conversor analógico-digital (ADC) del microcontrolador y su aplicación en la lectura de un sensor LDR. Además, se logró implementar con éxito la comunicación con la tira de LEDs WS2812 y el control de un servomotor mediante señales PWM, integrando varios periféricos de manera coordinada.

Entre las posibles mejoras, se destaca la necesidad de optimizar la comunicación por USART, la cual limitó la velocidad de respuesta del sistema. También sería conveniente incorporar un modo de calibración que permita al usuario registrar valores de referencia personalizados para cada color, adaptando el sistema a diferentes condiciones de iluminación. Finalmente, el uso de un servomotor de 360° posibilitaría cubrir un rango mayor de colores en la rueda.

V-C. Piano

Este módulo permitió afianzar el manejo de interrupciones, temporizadores y la generación de señales de distinta frecuencia mediante la modulación por ancho de pulso. A partir de ello, se consiguió la reproducción tanto de notas individuales como de melodías almacenadas en memoria.

Como mejoras futuras, se propone ampliar el repertorio de canciones predefinidas y aumentar la cantidad de notas disponibles en el teclado, extendiendo el rango tonal del instrumento. Asimismo, podría trabajarse en la sincronización de las pistas para reducir el leve desfase que aparece durante la reproducción prolongada.

V-D. Cerradura

El desarrollo de la cerradura electrónica permitió integrar múltiples conceptos del curso: el uso de la memoria EEPROM para el almacenamiento persistente de contraseñas, la lectura de teclados matriciales con técnicas de *debouncing*, la creación de máquinas de estados y la gestión de tareas temporizadas mediante un solo temporizador.

Como mejoras, se plantea incorporar un botón que permita visualizar temporalmente la contraseña antes de confirmar, mejorando la experiencia del usuario sin comprometer la seguridad. Además, la inclusión de un solenoide o mecanismo físico real de bloqueo daría una respuesta tangible al proceso de autenticación, completando el sistema de manera práctica.

V-E. Reflexión general

De manera global, el laboratorio permitió integrar conceptos de electrónica digital, control en tiempo real y diseño de interfaces embebidas, aplicados a problemas concretos de adquisición de datos, procesamiento y control. El trabajo desarrollado evidenció el funcionamiento interno del microcontrolador ATmega328P y su ecosistema de periféricos, así como la importancia del diseño modular, la depuración estructurada y la optimización de recursos en sistemas embebidos.

REFERENCIAS

- [1] A. S. Sedra and K. C. Smith, *Microelectronic Circuits*, 8th ed. Oxford University Press, 2021, conceptos de fotoconductividad y respuesta espectral en materiales semiconductores.
- [2] Microchip Technology Inc. Atmega328p datasheet. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf
- [3] M. A. Mazidi, S. Naimi, and S. Naimi, *The AVR Microcontroller and Embedded Systems: Using Assembly and C*. Pearson Education, 2014, referencia clásica sobre arquitectura AVR, ADC, PWM y EEPROM.

- [4] J. Angeles, *Fundamentals of Robotic Mechanical Systems: Theory, Methods, and Algorithms*, 5th ed. Springer, 2018, referencias sobre control por PWM y cinemática de actuadores.
- [5] International MIDI Association, *MIDI 1.0 Detailed Specification*, IMA, 1996, documento técnico que define la codificación de eventos musicales en el formato MIDI.
- [6] E. Williams, *Make: AVR Programming: Learning to Write Software for Hardware*. O'Reilly Media, 2014, guía práctica de programación en C para microcontroladores AVR.
- [7] J. J. Labrosse, *Embedded Systems Building Blocks: Complete and Ready-to-Use Modules in C*. CRC Press, 2013, base teórica sobre máquinas de estados finitos y multitarea cooperativa en sistemas embebidos.
- [8] AVR Libc Development Team, *AVR Libc User Manual, Version 2.2.0*, Savannah GNU Project, 2024, documentación oficial de la biblioteca estándar de C para AVR. [Online]. Available: <https://www.nongnu.org/avr-libc/user-manual/index.html>
- [9] ShivamJoker, "Midi to arduino converter," <https://arduinomidi.netlify.app/>, 2024, herramienta en línea para convertir archivos MIDI a código Arduino.
- [10] circuito.io. (2018) Arduino uno pinout diagram. [Online]. Available: <https://www.circuito.io/blog/arduino-uno-pinout/>
- [11] ARXterra. Addressing modes — 8-bit avr instruction set. [Online]. Available: <https://www.arxterra.com/3-addressing-modes/>
- [12] Software Particles. (2024, Apr.) Learn how an 8x8 led display works and how to control it using an arduino. Figura: 8x8 LED matrix pin mapping (imagen del artículo). [Online]. Available: <https://softwareparticles.com/learn-how-an-8x8-led-display-works-and-how-to-control-it-using-an-arduino/>
- [13] Carpeta del laboratorio (google drive). Carpeta compartida del laboratorio. [Online]. Available: <https://drive.google.com/drive/u/0/folders/1fP0aILozXcapRgDPDNWT1TRhAYr1PPPT>
- [14] Microchip Community (AVR Freaks). Avr freaks — comunidad de desarrolladores avr. Foros técnicos y soluciones prácticas sobre AVR. [Online]. Available: <https://www.avrfreaks.net/>
- [15] J. Ganssle. A guide to debouncing. Referencia clásica para anti-rebote en pulsadores/encoders. [Online]. Available: <https://www.ganssle.com/debouncing.htm>
- [16] G. Schmidt. (2021, Sep.) Beginners introduction to the assembly language of atmel-avr-microprocessors. Tutorial de avr-asm-tutorial.net (revisión septiembre 2021). [Online]. Available: <https://kitsandparts.com/tutorials/assemblers/BeginnersAVRasm.pdf>
- [17] T. Redelberger. (2019, Apr.) Avr assembler for complex projects. Versión 0.4 (2019-04-06). [Online]. Available: https://web222.webleient5.de/doc/swdev/avrasm/advavrasm2/AdvancedUseAVRASM2_en_20190406.pdf
- [18] Laboratorio de Microcontroladores, UTEC, "Repositorio de laboratorio de microcontroladores (tec.micro)," <https://github.com/MateoLecunaTec.Micro>, 2025, accedido el 26 de septiembre de 2025.

APÉNDICE

A. Drive de Laboratorio

La carpeta de Google Drive correspondiente a este laboratorio puede encontrarse en https://drive.google.com/drive/folders/1fylHKdJC3ad7wB_ZZz2fyQ-oS7G_49QM?usp=sharing. Allí encontrará evidencias del trabajo realizado durante esta actividad.

B. Fragmentos de código

B1. Plotter: —

B1a. Configuración de USART: —

```
#define F_CPU 16000000UL // Frecuencia del
// reloj del micro (16 MHz)
#define BAUD 9600 // Velocidad de
// transmisión (baudios)
#define BRC ((F_CPU / 16 / BAUD) - 1) // Valor
// para UBRR
#define TX_BUFFER_SIZE 128
```

```
#define RX_BUFFER_SIZE 128 void appendSerial(char
c)
{
    serialBuffer[serialWritePos] = c;
    serialWritePos++;

    if (serialWritePos >= TX_BUFFER_SIZE) {
        serialWritePos = 0; // wrap-around
    }
}

void serialWrite(const char *s){
    for (uint8_t i = 0; i < (uint8_t)strlen(s); i
    ++){
        serialBuffer[serialWritePos] = s[i];
        serialWritePos = (serialWritePos + 1) %
        TX_BUFFER_SIZE;
    }
    UCSR0B |= (1 << UDRIE0); // habilita ISR
    UDRE
}

ISR(USART_UDRE_vect){
    if (serialReadPos != serialWritePos){
        UDR0 = serialBuffer[serialReadPos];
        serialReadPos = (serialReadPos + 1) %
        TX_BUFFER_SIZE;
    } else {
        UCSR0B &= ~(1 << UDRIE0); // nada mas que
        enviar
    }
}

char peekChar(void)
{
    char ret = '\0';

    if (rxReadPos != rxWritePos)
    {
        ret = rxBuffer[rxReadPos];
    }

    return ret;
}

char Chardos(void)
{
    char ret = '\0';

    if (rxReadPos != rxWritePos)
    {
        ret = rxBuffer[rxReadPos];

        rxReadPos++;

        if (rxReadPos >= RX_BUFFER_SIZE)
        {
            rxReadPos = 0;
        }
    }

    return ret;
}

ISR(USART_RX_vect)
{
    rxBuffer[rxWritePos] = UDR0;

    rxWritePos++;

    if (rxWritePos >= RX_BUFFER_SIZE)
    {
        rxWritePos = 0;
    }
}
```

Listing 1. Configuración de USART

B1b. Variables globales: —

```

// Direcciones base
#define SOLENOIDDOWN 1
#define SOLENOIDUP 2
#define DOWN 3
#define UP 4
#define RIGHT 5
#define LEFT 6
#define STOP 7

// Diagonales
#define UPRIGHT 8
#define UPLEFT 9
#define DOWNRIGHT 10
#define DOWNLEFT 11

void apagar(void) {

PORTD = (PORTD & 0b00000011) | 0b00001000;

}

void apagar2(void) {

PORTD = (PORTD & 0b00001111) | 0b00000000;

}

void Subir_s(void)
{
PORTD = (PORTD & 0b00000011) | 0b00000100;

}

void Bajar(void)
{
PORTD = (PORTD & 0b00001111) | 0b00010000;

}

void Subir(void)
{
PORTD = (PORTD & 0b00001111) | 0b00100000;

}

void Izquierda(void)
{
PORTD = (PORTD & 0b00001111) | 0b01000000;

}

void Derecha(void)
{
PORTD = (PORTD & 0b00001111) | 0b10000000;

}

void AbajoIzquierda(void)
{
PORTD = (PORTD & 0b00001111) | 0b01010000; //
D6 y D4

}

void AbajoDerecha(void) {
PORTD = (PORTD & 0b00001111) | 0b10010000; //
D6 y D4

}

void ArribaIzquierda(void)
{
PORTD = (PORTD & 0b00001111) | 0b01100000; //
D3 y D4

}

void ArribaDerecha(void) {
PORTD = (PORTD & 0b00001111) | 0b10100000; //
D6 y D4

```

```

}

```

Listing 2. Variables globales

B1c. Menu de pruebas: —

```

void peurbas(char c){
switch (c) {
case 'x': apagar(); break;
case 's': Subir_s(); break;
case 'u': Subir(); break;
case 'd': Bajar(); break;
case 'l': Izquierda(); break;
case 'r': Derecha(); break;
case 'z': AbajoIzquierda(); break;
case 'c': AbajoDerecha(); break;
case 'q': ArribaIzquierda(); break;
case 'e': ArribaDerecha(); break;
case 'j': centrar(); break;
default: break;
}
}

```

Listing 3. Menu de pruebas

B1d. Calculo trigonometrico: —

```

// ---- Generar tablas seno/cos 0-90 ----
void precompute_semicirculo_arrays_90(uint16_t
I_ms, uint16_t tR90[90], uint16_t tU90[90]) {
for (uint16_t i = 0; i < 90; ++i) {
float th = (float)i * (float)M_PI / 180.0f
;
float c = cosf(th);
float s = sinf(th);
if (c < 0) c = 0; else if (c > 1) c = 1;
if (s < 0) s = 0; else if (s > 1) s = 1;
tR90[i] = (uint16_t)lrintf(c * (float)I_ms
);
tU90[i] = (uint16_t)lrintf(s * (float)I_ms
);
}
}
// ---- Dibujar circulo completo ----
void Hacer_circulo(void) {
const uint16_t I_ms = 90; // Duracion
base de cada paso
uint16_t tR90[90], tU90[90]; // Tablas seno/
cos de 0 90

precompute_semicirculo_arrays_90(I_ms, tR90,
tU90);

delay_ms_timer1(100);
cli();

Subir_s(); // Baja el lapiz

for (uint16_t ang = 0; ang < 360; ++ang) {
uint16_t q = ang / 90; // Cuadrante
actual (0 3)
uint16_t idx = ang % 90; // indice
dentro del cuadrante (0-89)

switch (q) {
case 0: // Q1: UP + RIGHT
Derecha();
delay_ms_timer1(tU90[idx]);
Subir();
delay_ms_timer1(tR90[idx]);
apagar2();

```

```

        _delay_ms(10);
        break;

    case 1: // Q2: DOWN + RIGHT
        Bajar();
        delay_ms_timer1(tU90[idx]);
        Derecha();
        delay_ms_timer1(tR90[idx]);
        apagar2();
        _delay_ms(10);
        break;

    case 2: // Q3: DOWN + LEFT
        Izquierda();
        delay_ms_timer1(tU90[idx]);
        Bajar();
        delay_ms_timer1(tR90[idx]);
        apagar2();
        _delay_ms(10);
        break;

    case 3: // Q4: UP + LEFT
        Subir();
        delay_ms_timer1(tU90[idx]);
        Izquierda();
        delay_ms_timer1(tR90[idx]);
        apagar2();
        _delay_ms(10);
        break;
    }
}

apagar(); // Apaga motores
sei();
delay_ms_timer1(200);
}

```

Listing 4. Calculo trigonometrico

B1e. Deteccion de contorno: —

```

#Google Drive
drive.mount('/content/drive')

# Ruta de la imagen
image_path = '/content/drive/MyDrive/Semestre 4/
Forma-Flor.png'

# Cargar la imagen
img = Image.open(image_path)
img_cv = cv2.cvtColor(np.array(img), cv2.
    COLOR_RGB2BGR)

# Pasar a escala de grises y suavizar
gray = cv2.cvtColor(img_cv, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0)

# Deteccion de bordes con Canny
edges = cv2.Canny(blur, 50, 150)

# Encontrar contornos
contours, _ = cv2.findContours(edges, cv2.
    RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

# Mostrar contornos detectados
img_with_contours = img_cv.copy()
cv2.drawContours(img_with_contours, contours, -1,
    (0, 255, 0), 3)
cv2.imshow(img_with_contours)

```

Listing 5. Deteccion de contorno

B1f. Trazado en txt: —

```

#
=====

# Generar secuencia de movimientos
#
=====

movimientos2 = []

if len(contours) > 0:
    for contour in contours:
        if len(contour) < 2:
            continue

        movimientos2.append("SOLENOID_DOWN") #
        Baja la bobina

        for i in range(len(contour) - 1):
            x1, y1 = map(int, contour[i][0])
            x2, y2 = map(int, contour[i + 1][0])
            dx = x2 - x1
            dy = y2 - y1

            if dx > 0 and dy == 0:
                direction = "RIGHT"
            elif dx < 0 and dy == 0:
                direction = "LEFT"
            elif dy > 0 and dx == 0:
                direction = "DOWN"
            elif dy < 0 and dx == 0:
                direction = "UP"
            elif dx > 0 and dy < 0:
                direction = "UPRIGHT"
            elif dx < 0 and dy < 0:
                direction = "UPLEFT"
            elif dx > 0 and dy > 0:
                direction = "DOWNRIGHT"
            elif dx < 0 and dy > 0:
                direction = "DOWNLEFT"
            else:
                direction = "STAY"

        movimientos2.append(direction)

        movimientos2.append("SOLENOID_UP") # Sube
        la bobina

# Guardar movimientos en archivo
with open("movimientos2.txt", "w") as f:
    for move in movimientos2:
        f.write(f"{move}\n")

print(f"Archivo 'movimientos2.txt' guardado
con {len(movimientos2)} pasos.")
else:
    print("No se detecto ningun contorno.")

#
=====

# Agrupar cada 15 elementos y agregar coma final
#
=====

with open("movimientos2.txt") as f:
    direcciones = [line.strip() for line in f if
        line.strip()]

```

```
nuevas_lineas = []

for i in range(0, len(direcciones), n):
    grupo = direcciones[i:i+n]
    linea = ",".join(grupo) + "," # agrega coma
    extra al final
    nuevas_lineas.append(linea)

# Guardar archivo final con comas
with open("direcciones_comas.txt", "w") as f:
    f.write("\n".join(nuevas_lineas))

print(f"Listo : Se escribieron {len(direcciones)}
    movimientos en {len(nuevas_lineas)} lineas.")

# Leer y mostrar contenido
with open("direcciones_comas.txt", "r") as f:
    content = f.read()

print(content)
```

Listing 6. Trazado en txt

B1g. Funcion de trazado: —

```
void ejecutar_forma(const uint16_t *tablaDir,
    uint16_t size) {
    uint8_t cmd;

    _delay_ms(1000); // Espera inicial

    for (uint16_t i = 0; i < size; i++) {
        cmd = pgm_read_byte(&tablaDir[i]);

        if (cmd == STOP)
            break;

        switch (cmd) {
            case UP:          Subir(); break;
            case DOWN:        Bajar(); break;
            case LEFT:         Izquierda(); break
        ;
            case RIGHT:       Derecha(); break;

            case UPLEFT:      ArribaIzquierda();
        break;
            case UPRIGHT:     ArribaDerecha();
        break;
            case DOWNLEFT:    AbajoIzquierda();
        break;
            case DOWNRIGHT:   AbajoDerecha();
        break;

            case SOLENOID_UP:  apagar(); break;
            case SOLENOID_DOWN: Subir_s(); break;

            default:           apagar(); break;
        }
        _delay_ms(150);
    }

    _delay_ms(1000); // Espera final
    apagar();
}
```

Listing 7. Funcion de trazado

B1h. Ejecutar Triangulo: —

```
void dibujar_triangulo(void) {
```

```
    Subir_s();
    _delay_ms(500);
    Bajar();
    _delay_ms(27000);
    ArribaDerecha();
    _delay_ms(12000);
    ArribaIzquierda();
    _delay_ms(12000);

    apagar();

}
```

Listing 8. Ejecutar Triangulo

B1i. Ejecutar Cruz: —

```
void dibujar_cruz(void) {
    Subir_s();
    _delay_ms(500);
    ArribaDerecha();
    _delay_ms(24000);
    AbajoIzquierda();
    _delay_ms(12000);
    ArribaIzquierda();
    _delay_ms(12000);
    AbajoDerecha();
    _delay_ms(24000);
    apagar();

}
```

Listing 9. Ejecutar Cruz

B2. Colores: —

B2a. Librerías utilizadas: —

```
#define F_CPU 16000000UL

#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
#include <string.h>
```

Listing 10. Librerías utilizadas

B2b. Lectura de colores: —

```
uint8_t led_state = 0;
uint16_t adc_sample[] = {0,0,0};

// Leer adc
uint16_t adc_read(uint8_t channel) {
    ADMUX = (ADMUX & 0xF0) | (channel & 0x0F);
    ADCSRA |= (1 << ADSC);
    while (ADCSRA & (1 << ADSC));
    return ADC;
}

// Establecer color del led rgb (iluminador)
void rgb_set(uint8_t r, uint8_t g, uint8_t b) {
    PORTB = (PORTB & ~(1 << RED) | (1 << GREEN) | (1
    << BLUE))) |
    ((r<<RED) | (g<<GREEN) | (b<<BLUE));
}

// Ciclo de medicion RGB
void rgb_read(void) {
```

```

switch(led_state){
    case 0:
        rgb_set(1,0,0);
        adc_sample[0] = adc_read(0); // Rojo
        break;

    case 1:
        rgb_set(0,1,0);
        adc_sample[1] = adc_read(0); // Verde
        break;

    case 2:
        rgb_set(0,0,1);
        adc_sample[2] = adc_read(0); // Azul
        break;
}
}

```

Listing 11. Lectura de colores

B2c. Convertidor a string: —

```

// Convierte un valor entero sin signo en un
// string
void UTOA(uint16_t value, char *buffer) {
    char temp[6];
    int i = 0, j = 0;

    if (value == 0) {
        buffer[0] = '0';
        buffer[1] = '\0';
        return;
    }

    // Convert digits to temp buffer (reversed)
    while (value > 0 && i < sizeof(temp) - 1) {
        temp[i++] = (value % 10) + '0';
        value /= 10;
    }

    // Reverse digits into final buffer
    while (i > 0) buffer[j++] = temp[--i];
    buffer[j] = '\0';
}

```

Listing 12. Convertidor de unsigned integer a string

B2d. Manejo de tira led WS2812: —

```

void send_bit(uint8_t bitVal){
    if(bitVal){
        PORTD |= (1<<LED_PIN);
        asm volatile (
            "nop\n\t""nop\n\t""nop\n\t""nop\n\t""nop\n\t"
            "\t"
            "nop\n\t""nop\n\t""nop\n\t""nop\n\t""nop\n\t";

        PORTD &= ~(1<<LED_PIN);

        asm volatile (
            "nop\n\t""nop\n\t""nop\n\t""nop\n\t";

        } else {
            PORTD |= (1<<LED_PIN);
            asm volatile (
                "nop\n\t""nop\n\t""nop\n\t";

            PORTD &= ~(1<<LED_PIN);
            asm volatile (
                "nop\n\t""nop\n\t""nop\n\t""nop\n\t""nop\n\t"
                "\t"

```

```

            "nop\n\t""nop\n\t""nop\n\t""nop\n\t""nop\n\t"
            "\t");
        }
    }

    // Envia un Byte a la tira de leds
    // Utiliza cli y sei para un timing preciso
    void send_byte(uint8_t byte) {
        cli();
        for (uint8_t i = 0; i < 8; i++) {
            send_bit(byte & 0x80); // Enviar msb
            primero
            byte <<= 1;
        }
        sei();
    }

    void ws2812_send_pixel(uint8_t r, uint8_t g,
        uint8_t b) {
        // Cambiar orden dependiendo de formato de
        // tira led o matriz
        send_byte(g);
        send_byte(b);
        send_byte(r);
    }

    // Encender n leds del mismo color
    void ws2812_fill(uint8_t r, uint8_t g, uint8_t b,
        uint16_t n) {
        cli();
        for (uint16_t i = 0; i < n; i++) {
            ws2812_send_pixel(r, g, b);
        }
        sei();
        ws2812_show();
    }

    // Establecer colores predeterminados
    void led_strip_set_color(uint8_t color_id) {
        uint8_t r = 0, g = 0, b = 0;

        switch (color_id) {
            case 1: // Rojo
                r = 255; g = 0; b = 0;
                break;

            case 2: // Amarillo
                r = 255; g = 100; b = 0;
                break;

            case 3: // Verde
                r = 0; g = 255; b = 0;
                break;

            case 4: // Azul claro
                r = 0; g = 255; b = 255;
                break;

            case 5: // Violeta
                r = 100; g = 0; b = 100;

                break;

            case 6: // Morado
                r = 200; g = 0; b = 75;

                break;

            case 7: // Blanco
                r = 255; g = 255; b = 255;
                break;

            default: // Apagar LED
                r = g = b = 0;
                break;

```



```

    }

    ws2812_fill(r, g, b, 50);
}

```

Listing 13. Manejo de tira led WS2812

B2e. Movimiento de servomotor: —

```

// Establecer angulo en el servomotor
void servo_set_angle(uint8_t angle) {
    uint16_t pulse = 1000 + ((uint32_t)angle *
    4000) / 180;
    OCR1A = pulse;
}

```

Listing 14. Movimiento de servomotor

B2f. Reconocimiento de colores: —

```

// Mapeado de colores RGB
typedef struct {
    const char *name;
    uint16_t r, g, b;
} ColorRef;

const ColorRef color_refs[] = {
    {"MORADO",    216, 157, 274},
    {"ROJO",      206, 149, 262},
    {"AMARILLO",  196, 99, 105},
    {"VERDE",     272, 192, 152},
    {"AZUL CLARO", 151, 153, 122},
    {"VIOLETA",   253, 275, 338},
    {"BLANCO",    110, 93, 91},
};

// Identificar color a partir de valores rgb.
const char* identify_color(
    uint16_t r,
    uint16_t g,
    uint16_t b
) {
    uint32_t best_dist = 0xFFFFFFFF;
    const char *best_name = "UNKNOWN";

    for (uint8_t i = 0; i < NUM_COLORS; i++) {
        int32_t dr = (int32_t)r - color_refs[i].r;
        int32_t dg = (int32_t)g - color_refs[i].g;
        int32_t db = (int32_t)b - color_refs[i].b;
        uint32_t dist = dr*dr + dg*dg + db*db;

        if (dist < best_dist) {
            best_dist = dist;
            best_name = color_refs[i].name;
        }
    }
    return best_name;
}

```

Listing 15. Reconocimiento de colores

B2g. Comunicación por USART: —

```

// Imprimir datos parseados por usart
void print_data(const char *color_name) {
    char buf[10];

    // Find by name

```

```

uint16_t ref_r = 0, ref_g = 0, ref_b = 0;
for (uint8_t i = 0; i < NUM_COLORS; i++) {
    if (strcmp(color_name, color_refs[i].name)
    == 0) {
        ref_r = color_refs[i].r;
        ref_g = color_refs[i].g;
        ref_b = color_refs[i].b;
        break;
    }
}

```

```

// Delta
int16_t dR = (int16_t)adc_sample[0] - ref_r;
int16_t dG = (int16_t)adc_sample[1] - ref_g;
int16_t dB = (int16_t)adc_sample[2] - ref_b;

// Printing
usart_write_str("Fotocelda: ");
UTOA(adc_sample[0], buf); usart_write_str(buf)
;
usart_write_str(" ");
UTOA(adc_sample[1], buf); usart_write_str(buf)
;
usart_write_str(" ");
UTOA(adc_sample[2], buf); usart_write_str(buf)
;

```

```

usart_write_str(" Color detectado: ");
usart_write_str(color_name);

```

```

usart_write_str(" Valor establecido: [");
UTOA(ref_r, buf); usart_write_str(buf);
usart_write_str(",");
UTOA(ref_g, buf); usart_write_str(buf);
usart_write_str(",");
UTOA(ref_b, buf); usart_write_str(buf);
usart_write_str("]");

```

```

usart_write_str(" Delta: [");
if (dR >= 0) usart_write_str("+");
else usart_write_str("-");
UTOA((dR >= 0) ? dR : -dR, buf);
usart_write_str(buf);
usart_write_str(",");

```

```

if (dG >= 0) usart_write_str("+");
else usart_write_str("-");
UTOA((dG >= 0) ? dG : -dG, buf);
usart_write_str(buf);
usart_write_str(",");

```

```

if (dB >= 0) usart_write_str("+");
else usart_write_str("-");
UTOA((dB >= 0) ? dB : -dB, buf);
usart_write_str(buf);
usart_write_str("]\r\n");
}

```

Listing 16. Comunicación por USART

B2h. Bucle principal: —

```

// programa principal
int main(void) {
    ws2812_init();
    usart_init();
    adc_init();
    rgb_init();
    servo_init();
    sei();

    while (1) {
        rgb_read();

```

```

        _delay_ms(50);

    }
}

```

Listing 17. Bucle principal

B3. Piano: —

B3a. Librerías utilizadas: —

```

#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <avr/pgmspace.h>

```

Listing 18. Librerías utilizadas

B3b. Guardado de notas musicales: —

```

#define G4 392
#define Fb4 370
#define E4 330
#define D4 294
// ...

```

Listing 19. Selección dinámica de prescaler

B3c. Selección dinámica de prescaler: —

```

void playFrequencyA(uint16_t freq) {
    if (!freq) return;

    DDRD |= (1 << PORTD6);

    // Elegir prescaler adecuado para esa
    frecuencia

    uint8_t presc_bits = 0;
    uint16_t ocr = 0;

    const uint16_t presc_list[] = {8, 64, 256,
    1024};
    const uint8_t bits_list[] = {0b010, 0b011, 0
    b100, 0b101};

    for (uint8_t i = 0; i < 4; i++) {
        ocr = (F_CPU / (2UL * presc_list[i] * freq
        )) - 1;
        if (ocr <= 255) {
            presc_bits = bits_list[i];
            break;
        }
    }

    TCCR0A = (1 << COM0A0) | (1 << WGM01);
    TCCR0B = presc_bits;
    OCR0A = (uint8_t)ocr;
}

// Dejar de reproducir frecuencia buzzer 1
void stopFrequencyA(void) {
    TCCR0A = 0;
    TCCR0B = 0;
    DDRD |= (1 << PORTD6);
    PORTD &= ~(1 << PORTD6);
}

```

Listing 20. Selección dinámica de prescaler

B3d. Reproducción de pistas: —

```

const int midiA[349][3] PROGMEM = {
    {G4, 252, 0},
    {Fb4, 252, 0},
    {E4, 252, 0},
    {E4, 252, 0},
    // ...
}

void read_midi_event(const int (*track)[3],
    uint16_t index, int out_values[3]) {
    for (uint8_t i = 0; i < 3; i++) {
        out_values[i] = pgm_read_word(&(track[
        index][i]));
    }
}

// Reproducir melodía o track en buzzer 1
void playTrackA(void) {
    int values[3];
    if (song == 0) {
        read_midi_event(midiC, indexA++, values);
    } else {
        read_midi_event(midiA, indexA++, values);
    }

    uint16_t freq = values[0];
    maxCountAon = values[1];
    maxCountAoff = values[2];

    playFrequencyA(freq);
    enableCountAon = 1;
}

ISR(TIMER1_OVF_vect) {
    TCNT1 = 65536 - 250; // 1ms preload

    // Debouncing
    if (debounce_active) { // 1 ms debounce
        PCICR |= (1 << PCIE1) | (1 << PCIE2);
        debounce_active = 0;
    }

    // Note playing
    if (enableCountAon) {
        if (++countA > maxCountAon) {
            eventAon = 1;
        }
    }
    else if (enableCountAoff) {
        if (++countA > maxCountAoff) {
            eventAoff = 1;
        }
    }

    if (enableCountBon) {
        if (++countB > maxCountBon) {
            eventBon = 1;
        }
    }
    else if (enableCountBoff) {
        if (++countB > maxCountBoff) {
            eventBoff = 1;
        }
    }
}

```

Listing 21. Reproducción de pistas

B3e. Manejo de estados por USART: —

```
// Manejo de cambio de estados de usart
void handleUSART(uint8_t character){
    if (character == '1'){
        mode = 1;
        eventAoff = 1;
        eventBoff = 0;

        song = 0;

        eventAon = 0; // Encender track A
        indexA = 0; // Posicion track A
        countA = 0; // Conteo de overflow de notas de track A
        maxCountAon = 0; // Maximo conteo de overflow encendido en A
        maxCountAoff = 0; // Maximo conteo de overflow apagado en A
        enableCountAon = 0; // Habilitar conteo de encendido en A
        enableCountAoff = 0; // Habilidad conteo de apagado en A

        eventBon = 0; // Encender track B
        indexB = 0; // Posicion track B
        countB = 0; // Conteo de overflow de notas de track B
        maxCountBon = 0; // Maximo conteo de overflow encendido en B
        maxCountBoff = 0; // Maximo conteo de overflow apagado en B
        enableCountBon = 0; // Habilitar conteo de encendido en B
        enableCountBoff = 0; // Habilidad conteo de apagado en B

        PCICR &= ~(1 << PCIE1) | (1 << PCIE2));
        stopFrequencyB();

    } else if (character == '2'){
        mode = 1;
        eventAoff = 1;
        eventBoff = 1;

        song = 1;

        eventAon = 0; // Encender track A
        indexA = 0; // Posicion track A
        countA = 0; // Conteo de overflow de notas de track A
        maxCountAon = 0; // Maximo conteo de overflow encendido en A
        maxCountAoff = 0; // Maximo conteo de overflow apagado en A
        enableCountAon = 0; // Habilitar conteo de encendido en A
        enableCountAoff = 0; // Habilidad conteo de apagado en A

        eventBon = 0; // Encender track B
        indexB = 0; // Posicion track B
        countB = 0; // Conteo de overflow de notas de track B
        maxCountBon = 0; // Maximo conteo de overflow encendido en B
        maxCountBoff = 0; // Maximo conteo de overflow apagado en B
        enableCountBon = 0; // Habilitar conteo de encendido en B
        enableCountBoff = 0; // Habilidad conteo de apagado en B

        PCICR &= ~(1 << PCIE1) | (1 << PCIE2));
        stopFrequencyA();
    }
}
```

```
    } else if (character == 'P'){
        mode = 0;
        eventAoff = 0;
        eventBoff = 0;

        eventAon = 0; // Encender track A
        indexA = 0; // Posicion track A
        countA = 0; // Conteo de overflow de notas de track A
        maxCountAon = 0; // Maximo conteo de overflow encendido en A
        maxCountAoff = 0; // Maximo conteo de overflow apagado en A
        enableCountAon = 0; // Habilitar conteo de encendido en A
        enableCountAoff = 0; // Habilidad conteo de apagado en A

        eventBon = 0; // Encender track B
        indexB = 0; // Posicion track B
        countB = 0; // Conteo de overflow de notas de track B
        maxCountBon = 0; // Maximo conteo de overflow encendido en B
        maxCountBoff = 0; // Maximo conteo de overflow apagado en B
        enableCountBon = 0; // Habilitar conteo de encendido en B
        enableCountBoff = 0; // Habilidad conteo de apagado en B

        startDebounceTimer();
        stopFrequencyA();
        stopFrequencyB();
    }
}
```

Listing 22. Manejo de estados por USART

B3f. Programa principal: —

```
// Programa principal
int main(void) {
    timer1_init();
    usart_init_9600();
    init_piano_buttons();
    sei();

    usart_write_str("Elija una opcion:\r\n");
    usart_write_str("[1] Dragon Ball - Cha-La Head-Cha-La\r\n");
    usart_write_str("[2] Portal - Still alive\r\n");
    usart_write_str("[P] Modo piano\r\n");

    while (1){
        if (mode == 0){
            piano_mode();
        } else if (mode == 1){
            song_mode();
        }
        uint8_t c;
        if (usart_read_try(&c)) {
            handleUSART(c);
        }
    }
}
```

Listing 23. Programa principal

B4. Cerradura: —

B4a. Librerías para lcd: —

```
#include "i2c_master.h"
#include "i2c_master.c"
#include "liquid_crystal_i2c.h"
#include "liquid_crystal_i2c.c"

int main(void){
    LiquidCrystalDevice_t device = lq_init(0x27,
    16, 2, LCD_5x8DOTS);
    lq_turnOnBacklight(&device);
    char welcomeText[] = "Bienvenido!";
    lq_print(&device, welcomeText);
    while (1);
}
```

Listing 24. Librerías utilizadas para pantalla LCD

B4b. Lógica de estados: —

```
typedef enum { UI_MENU, UI_INGRESO,
    UI_CAMBIO_ACTUAL, UI_CAMBIO_NUEVA, UI_ABIERTO,
    UI_ALARMA } ui_state_t;

int main(void){
    while(1){
        char key = keypad_scan();
        if (key) {
            switch (ui_state)
            {
                case UI_MENU: break;
                case UI_INGRESO: break;
                case UI_CAMBIO_ACTUAL: break;
                case UI_CAMBIO_NUEVA: break;
                case UI_CAMBIO_ALARMA: break;
                case UI_CAMBIO_ABIERTO: break;
            }
        }
    }
}
```

Listing 25. Lógica de estados

B4c. Lectura multiplexada de keypad: —

```
// Mapeo de keypad
const char keypad[4][4] = {
    {'1', '2', '3', 'A'},
    {'4', '5', '6', 'B'},
    {'7', '8', '9', 'C'},
    {'*', '0', '#', 'D'}
};

char keypad_scan(void) {
    if (!keypad_enable) return 0;

    uint8_t row, col;
    uint8_t cols;
    static uint8_t prevKey; // store for later

    for (row = 0; row < 4; row++) {
        PORTD = (PORTD | 0xF0) & ~(1 << (row + 4));
        ;
        _delay_us(5);
        cols = PIND & 0x0F;

        for (col = 0; col < 4; col++) {
            if (!(cols & (1 << col)) ) {
                if ((prevKey == keypad[row][col]))
                    return 0;
            }
        }
        prevKey = keypad[row][col];
        return keypad[row][col];
    }
    keypad_debounce_ms(200);
}
```

```
        prevKey = keypad[row][col];
        return keypad[row][col];
    }
}
prevKey = 0;
return 0;
}
```

Listing 26. Función de lectura multiplexada de keypad

B4d. Implementación de tareas: —

```
uint32_t millis_counter = 0;

uint32_t keypad_on_at = 0;
uint8_t keypad_enable = 1;

ISR(TIMER0_OVF_vect){
    millis_counter++;
}

uint32_t millis_now(void) {
    uint32_t m;
    cli(); // disable interrupts
    m = millis_counter;
    sei(); // re-enable
    return m;
}

void keypad_debounce_ms(uint16_t delay_ms){
    keypad_enable = 0;
    keypad_on_at = millis_now() + delay_ms;
}

void keypad_task(void){
    if (!keypad_enable && (millis_now() >
    keypad_on_at)){
        keypad_enable = 1;
    }
}

int main(void){
    // main code ...
    while (1){
        keypad_task();
        // loop code ...
    }
}
```

Listing 27. Implementación de tareas

B4e. Escritura y lectura de EEPROM: —

```
#include <avr/eeprom.h>

#define MAX_PASSWORD_LENGTH 6
#define EEPROM_MAGIC 0x42

uint8_t EEMEM ee_magic;
char EEMEM ee_password[MAX_PASSWORD_LENGTH + 1];

char storedPassword[MAX_PASSWORD_LENGTH + 1] = "
123456";
char typedPassword[MAX_PASSWORD_LENGTH + 1];

void eeprom_load_password(void) {
    if (eeprom_read_byte(&ee_magic) !=
    EEPROM_MAGIC) {
        eeprom_update_block(

```

```

        storedPassword,
        ee_password,
        sizeof(storedPassword)
    );

    eeprom_update_byte(&ee_magic, EEPROM_MAGIC
);
    } else {
        eeprom_read_block(
            storedPassword,
            ee_password,
            sizeof(storedPassword)
        );
    }
}

void eeprom_save_password(const char *pwd) {
    char buf[MAX_PASSWORD_LENGTH + 1];
    strncpy(buf, pwd, MAX_PASSWORD_LENGTH);
    buf[MAX_PASSWORD_LENGTH] = '\0';
    eeprom_update_block(buf, ee_password, sizeof(
buf));
}

```

Listing 28. Escritura y lectura de EEPROM

B4f. Tarea de alarma: —

```

void alarm_task(void) {
    if (!alarm_active) return;
    uint32_t now = millis_now();

    if (now > alarm_until) {
        alarm_active = 0;
        PORTB &= ~(1<<PORTB5);
        led_red_on();
        return;
    }

    if (now > alarm_next_toggle) {
        alarm_next_toggle = now + ALARM_TOGGLE_MS;
        if (alarm_phase) {
            led_red_on();
        } else {
            led_green_on();
        }
        alarm_phase ^= 1;

        buzzer_beep(100);
    }
}

```

Listing 29. Tarea de alarma